Class 310

22L-6946 Fatima Imran

19/8/24 Lec#1

1) OOP ↙ SDA ↘ 2) Design Pattern

- Objectoriented paradigm (OO)
  - → abstraction
  - → encapsulation
  - → inheritence
  - → polymorphis

2.x version of UML → usage for modeling.

1) requirements of project → clients (through surveys & questionares)

2) modeling • digrams will show different aspect of software (may be ① dynamic or ② static)

Design (front-end + backend)

- modularity (different independent modules)
- Separation of concerns
- reusability

SDA helps software to be

- flexible
- reusable
- maintainable

FRM

CRC Card — Class responsibility collabrative card

21/8/24 Lec#2

Software Engineering : → solving consumer's/customers problem.

Stakeholders : • users — UCSD

ordering, paying ← • customers (clients)

develop & maintain ← • software developers

• Development managers → run the organization

SRS document → (2) Planning ← teams

(Software requirement specification)

SDLC

① Requirements analysis — all stakeholders are involved.

Software development lifecycle.

↓ methodology

⑥ phases.

③ Architectusal design → figma

④ Software development → coding

⑤ Testing :

⑥ Deployment :

maintenance is most crucial and time-taking

Programs organized as collection of objects.

OOP :- to understand the software in terms of objects.

classes defines the responsibilities of objects.

• information passing, hide & show → relationships

inheritance.

OOP Design :-

Object-oriented decomposition : breakdown system in terms of objects, it helps diff level of modeling — static, dynamic, logical, physical.

static → show structure of system and don't show changes

dynamic → if such event occurs then system can be modified like what.

physical → e.g API layers.

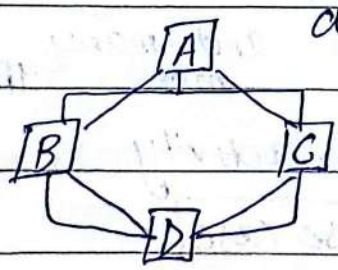
conceptional framework.

OOA (Analysis) → requirements. of classes & objects in problem domain

Object Model: 4 elements

1) Abstraction → show specific to problem things only → hide unnecessary stuff.
   - abstract classes are used → we use pure virtual functions.
2) Encapsulation → private data members, → we provide public methods to manipulate but don't provide direct access.
3) Modularity → Separation of concerns, related or similar information is placed in one module, grouping
   - → helps in data testing, reusability, managing
   - → interdependence btw modules should be least.
   - → make diff classes, files, header files. which causes less interdependence and information hiding occurs.
4) Hierarchy → means ordering. tells organization, structure. Mostly happens through aggregation (has-a) inheritence (is-a)
   - ↳ single
   - ↳ multiple (more than 1 parent classes)

if no implementation detail.



→ multiple instances of display() so ambiguity.

whole-part relationship
part of some class is included in other class.

diamond problem can exist

→ use virtual functions
→ scope resolution :: manually resolve.
B::display();

Object Oriented Methodology:

(i) System conception → conceive requirements

(ii) Analysis → restate requirements by constructing models, understand the problem with detail

(iv) System design → architecture?

(v) Class design

(vi) Implementation

Analysis model

domain model → class diagram → divide system into concepts

application model → actual structure of system.

Lec#3

26/8/24

UML → Universal Unified Modelling language → for large scale projects

to provide a common language or platform to understand things.

→ Uml is also known as universal language

Static → structure → class, component and many more types.

functions → behavorial diagram → activity, use case, interaction

(dynamic nature of system)

↑ drive iteration planning.

Use-case : tells the functionality and feature of system e.g upload, edit pfp
→ step by step interaction of user and system
→ which features of system a user can use.
→ Also tell the effect of interaction with user.
→ only cater Functional Requirement (i)
↓ tells about system's working, features

(ii) Non-functional: constraints e.g *** on password.
↳ How response time 10s.

* we develop system iteratively
SDLC (software development cycle) → 1 iteration → a new version.
first whatsapp version → first iteration
next version → 2nd iteration

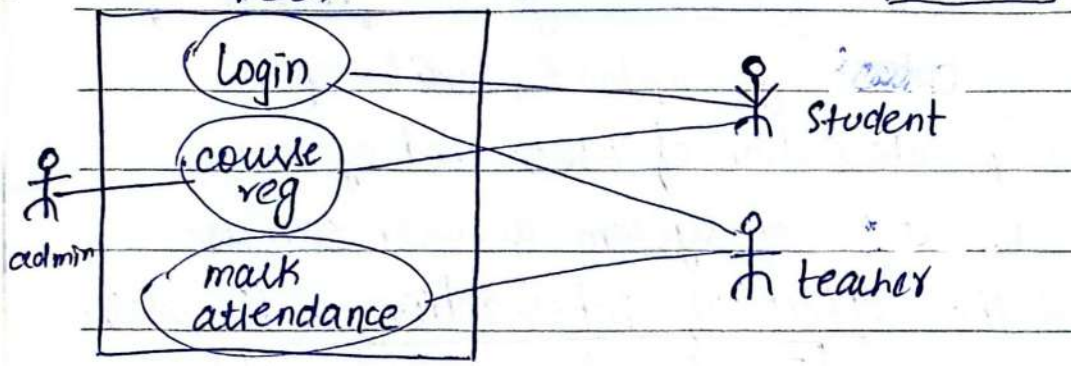
use case tells who who uses system → NOT → implementation specific language.
admin, user, teacher functionalities
Step by step in use case (table) description not in digram.
→ NOT → details about interfaces and screens.

System Boundary: → tells about things which should be implemented in system, already implemented.
→ first write system name outside of box (boundary), or inside
→ students 🧍, teacher 🧍 (human)
→ <<actor>> organization
→ functionality use case ⬭ (oval shape)

right side

FLEX

left side



(i) **Primary actors:** initiate system functionality (left side) — perform function.

(ii) **Secondary actor:** Support functionality asked by Primary actor. (right side)

Actor can be represented as

- inside of system or function eg CAPTCHA cannot be actor.
- stick/ (human figures)
- <<actor>> NADRA — external systems. e.g printers (can put devices).

error display → use case description.

(valid)

useful use-case:

↳ Boss Test — value? good use?

- a person can perform at one time and data remains consistent → EBP Test (Elementary Business Process). ↳ a managable piece. (normal size)
- → Size Test ↳ e.g add, delete, update ... into [manage product]

use case: → singular (not plural)

↳ write in verb, noun, phrases → Proper name. (e.g Fatima)
→ not a name

Example:

Req #1: adm create new blog acc

Content Management system

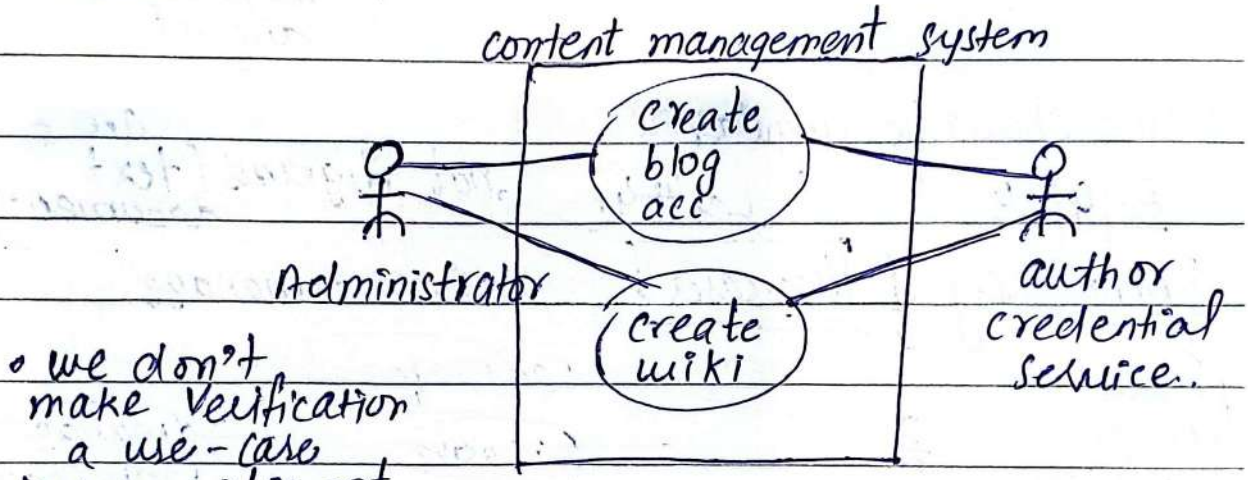
Actor: Admin (Primary)

Author Credential service (secondary)

Use cases: Create Blog account

↳ Create wiki

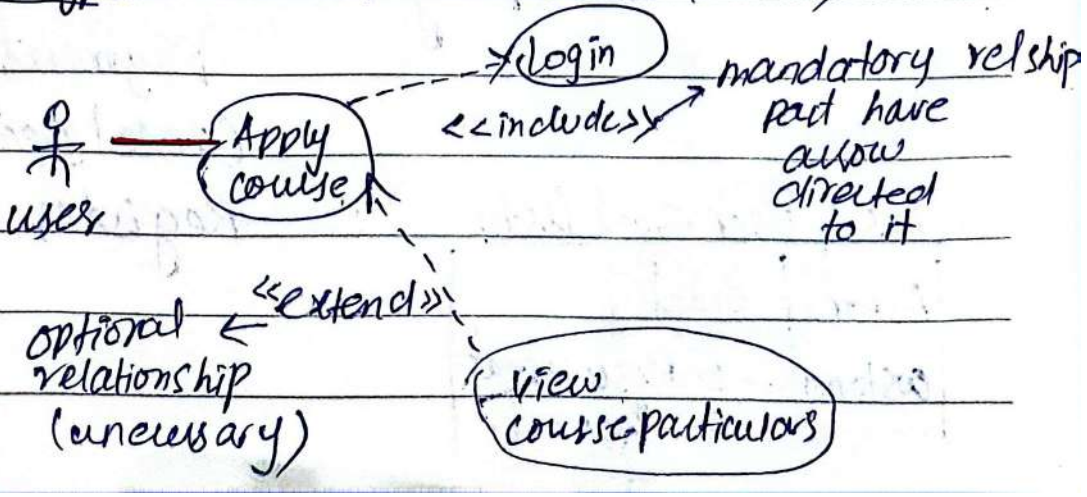
Req #2:



- we don't make verification a use-case because it's not good or efficient.

Association: between use-cases and users.

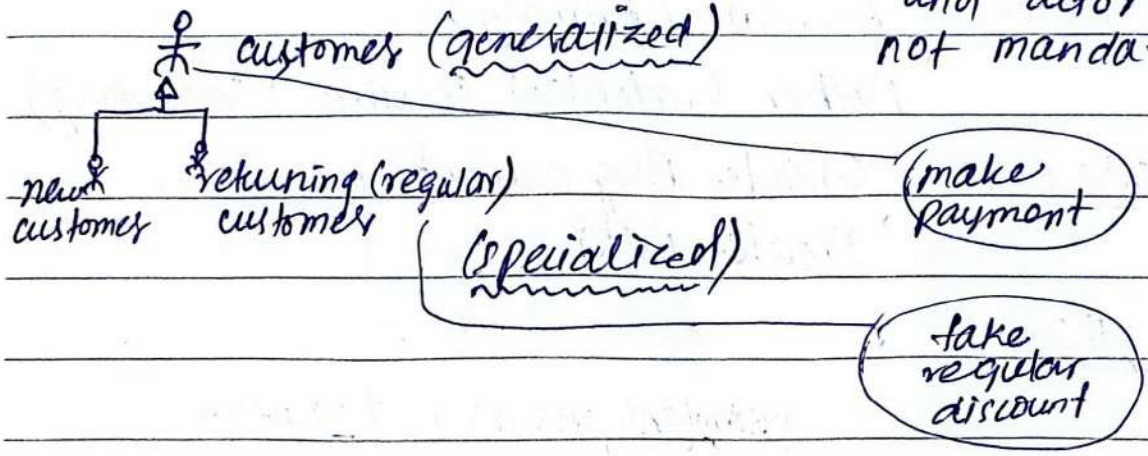
Stereotypes: between use-cases. (---->)



→ include & extend is discouraged cuz it increases complexity.

Generalization: (——▷) ⇒ association can happen btw extend and actor but not mandatory

used in inheritence

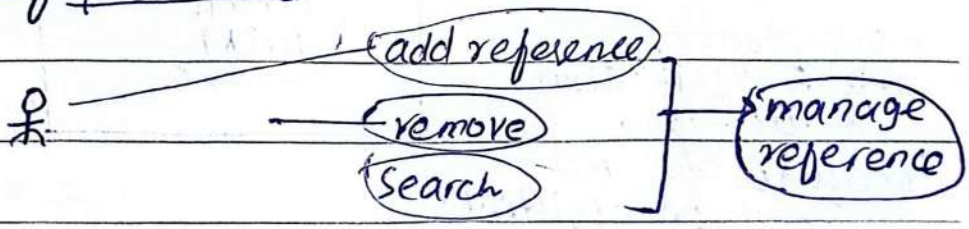


* name should be singular

28/8/24 Lec#4: → not digrams (are text documents)

Granularity of Use cases: CRUD → manage

↓ provide enough detail into one use case.



example:

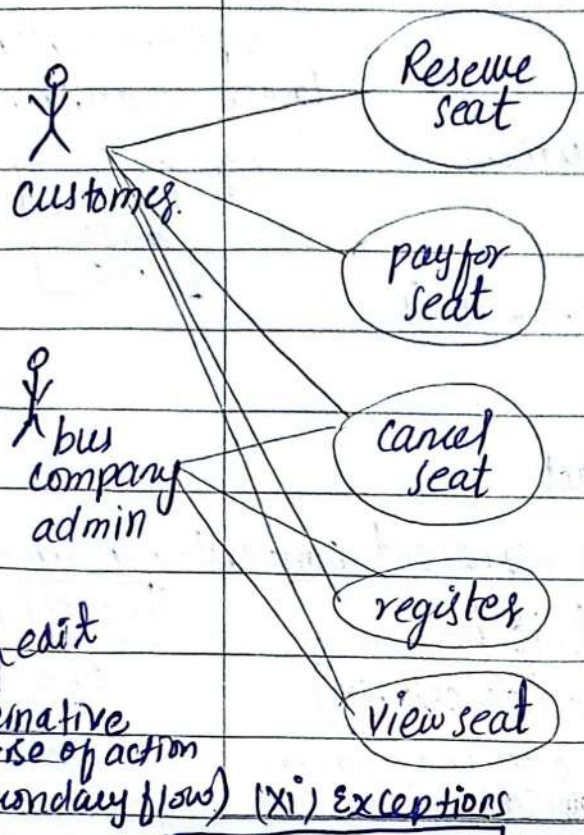
case: Actor: customer (admin), bus company

Use case:
Reserve seat
Payment
Cancel seat
Register

Actors & User Goal list:

Actor	Goal
customer	1. Reserveseat 2. Payment ....

Bus reservation System



(x) Alternative course of action (secondary flow)

(xi) Exceptions

Use case Description Table:

(i) Identifier e.g UC-01
(ii) Name
(iii) Purpose
(iv) Priority e.g high/medium/low
(v) Actors
(vi) Preconditions
(vii) Post conditions.
(viii) Dependencies
(ix) Typical course of action (primary flow)

2/9/24

Visual Dictionary

Check code quality ← Sonar Qube

Domain Modal: Lect#5:

Problem problem's domain

share stories, post ← concept, entities, ideas, relationships.

conceptual classes.

→ domain object model also known as
→ analysis object model.

(i) analysis class diagram

only concepts.

real world

it provides foundation

(ii) design class diagram.

it is at design level.
tells about Software implementation.

To represent domain model, we have

Analysis class diagram → box

upper → class name
lower → attributes.

analysis level
↓
class and its attributes.

e.g

Sale
date
time

no methods or software, it is at design level.

concept: idea, object

↳ symbol: how to represent concept, in image or [word] in case of classes.

↳ Intention: what it is doing (description)

optional

→ Extension: all sale class maps onto. e.g Sale

examples. Sale

problem space

Why domain model: It gives conceptual framework; helps you think, glossary of terms - noun based.

→ in order to know noun & vocabulary so we can easily make classes.

→ No direction mean arrow cause it bidirectional (—)

Association → strong relationship, have a label → bidirectional
* optional reading arrow

e.g Student * [Registers] * Courses
1..1 ▶ N → many.

define.

Multiplicity / Cardinality: one class invokes how many instances of other classes.

→ used in rare cases.

Dependency: A class can change another class.

↳ weaker relation

↳ temporary, it exists for some time, can exist without eachother.

dependent class. ↘

[order] - - - → [Payment Service]

▶ order only need it at the time of payment

no cardinality or label needed | represented by dotted line. (dependency)

Generalization: parent class (superclass)
child class (subclass)

↳ hollow arrow (→▷) ↳ type

Aggregation / Composition: → can be shown as simple association

Aggregation: contains / contained. hollow diamond ◇ on whole part

Composition: has-a dependency but not too much, it (a part) can exist without host. (independent existence of both.

* many
1..* one-to-many.
if not written then consider 1
1..40. one to forty
5 exactly 5
5,3,8 exactly 3,5,8

Composition: ◆ (full diamond)

↳ part class cannot exist without whole

if container class not exist contained class cannot exist.

find conceptual class

1) Reuse Modify existing Models.

2) Use a category list. ↘

e.g booking of seat → transaction (in the list)
(so it can be a class.)

↓

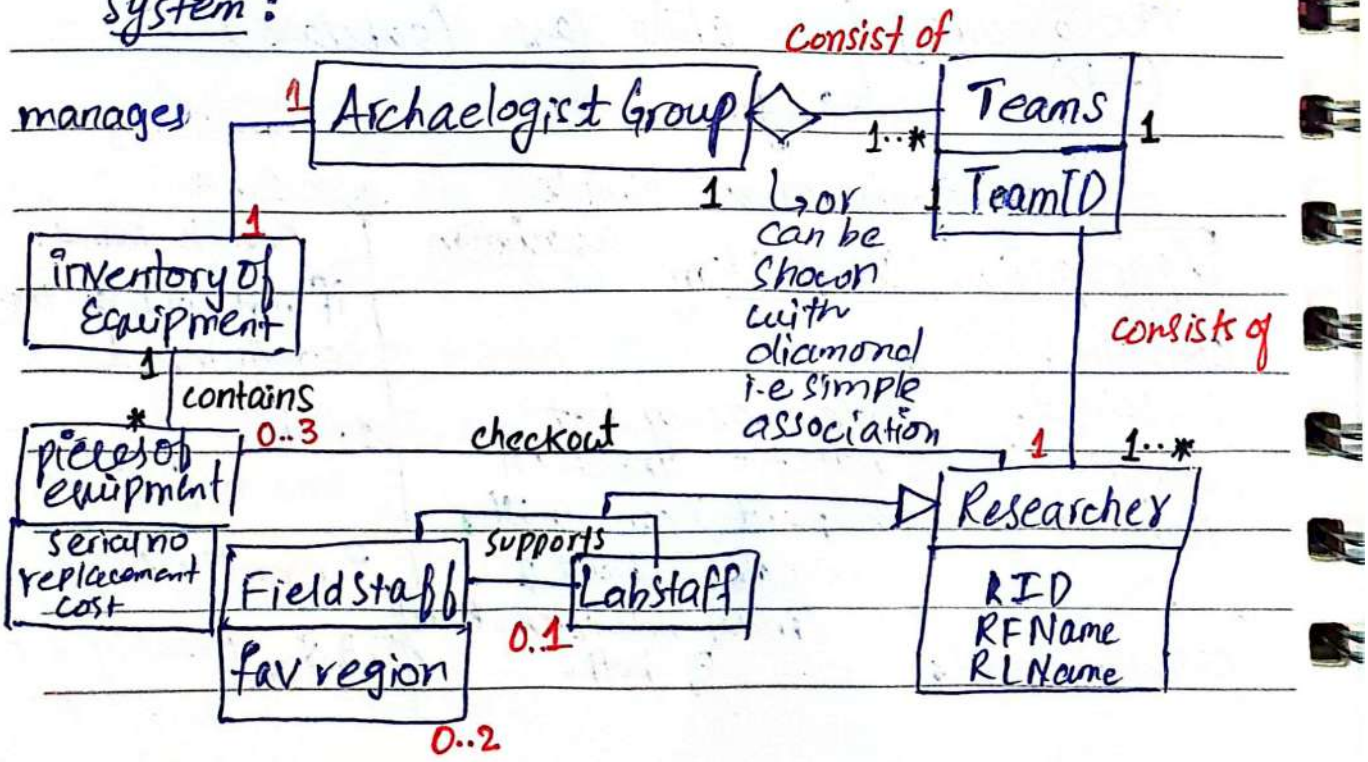
ticket → transaction item

3) Linguistic analysis → Nouns and Noun Phrases (extract in textual descriptions.)

↳ which most probably will be classes.

example:

Archaeologist Management System:



Association → name & it's multiplicity / cardinality is very important Lec#6. 9/9/24

navigational arrow (optional)

0..1
only 1 digit diff
0,1
2..5

Domain modeling guidlines:

1) name of territory (use existing names)
2) put relevant information only delete irrelevant
3) do not show things that donot exist.

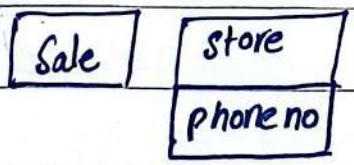
POST vs Register ✓ so that stake holders can understand (generic term)

store entity body in web server.

(class)

Concept vs Attribute → when anything can be represented by number or string

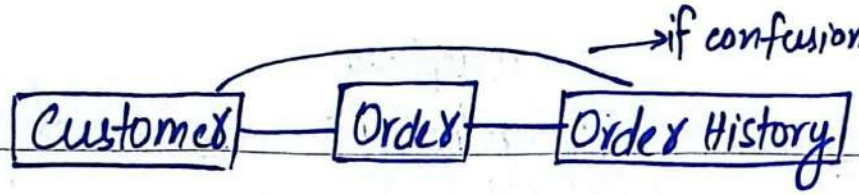
or



We have to put further information init

Association confusion → Then donot add. because it increases complexity and means dependency greater

⇒ we want less ∴ less association between class (minimal).



e.g CourseMgmSystem has no relation with lecturer's leave & research.

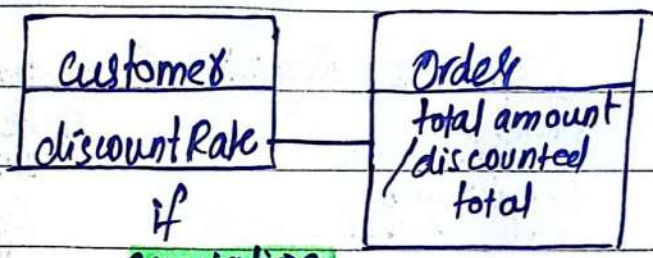
Derived attribute: calculated from already existing attribute or entity.

derived attribute.

(/) represented by forward slash

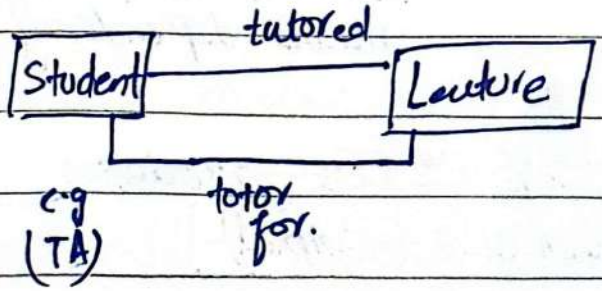
can exist in same class or another class

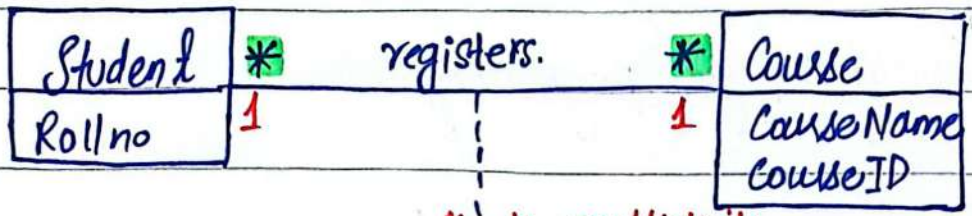
e.g



if association, then it will be derived else not

* Two classes can have multiple associations if diff or multiple relationships





lifetime dependency on association

related to relationship

e.g Grade is attribute of relationship it is not a single attribute of any class

consist pointer of both classes too.

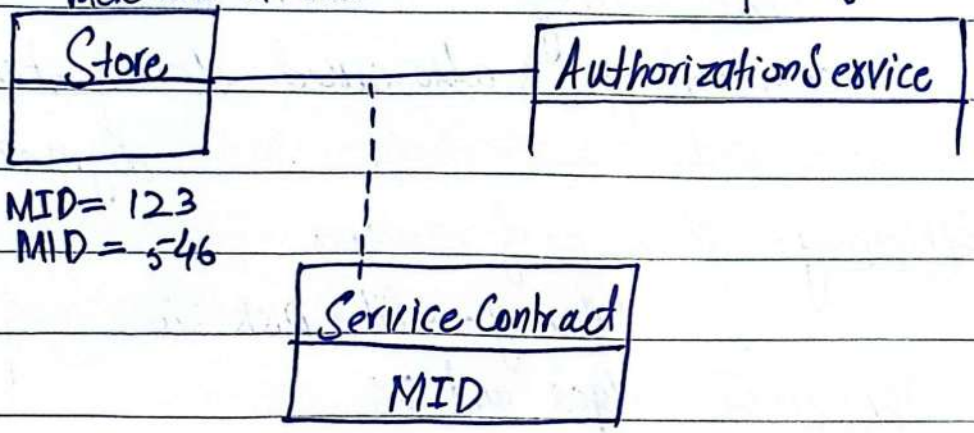
Association-class

association + relationship

⇒ Mostly in many to many

Merchant id = Mid.

e.g Jazz, Raast



if in a class, there are multiple values of same kind of attribute. Then place it in other class associated with another.

11/9/24

Qualities of good programming code

① Structure → proper indentation
commenting for understanding
consistent naming convention

- Camel case — helloBitch
- Snake case — A_B

② Maintainability:
↓ modularity — documentation
commenting is also called documentation of code
↓ digrams

③ Efficiency: efficient use of resources
Memory, CPU, Disk I/O
appropriate algos and DS

→ Avoid Redundancy: no repeated or unnecessary code.
reusability makes code look cleaner or better.

④ Scalability: → handle (easily) more data, users or processes if required in future

⑤ Robustness: → unlikely to break or failure
↳ error or exception handling. to prevent crashes or failure.

Input validation
↳ check input form

⑥ Testability:

↳ if modules → easy testing

consistency → same or good on multiple platforms or applications.

⑦ Simplicity:

↳ avoid overengineering, don't make things complex

Single responsibility principle → should do one work make multiple functions.

⑧ Reusability: Functions, classes, modules

⑨ Security:

↳ injection attacks: allow unauthorized data in to system.

→ Buffer overflow:

↳ fixed box / block of mem

→ Improper handling of sensitive data: → encryption

↳ https

⑩ → adaptability:

easy to extend and modify.

System is keen to change.

Activity Diagram:

(i) object nodes — class, data, [ ]

(ii) action nodes — defines activity

Action1 — activity node or action box

(iii) control nodes

(i) initial — tells who should be given control

(ii) final

(i) activity final

(ii) flow final

do not maintain sequence

undeterministic path

(multiple conditions)

True, Error

initial state

final state → bull's eye

outgoing arrows only

initial state → check tank level → final state ← incoming arrows

activity start

activity final

Flow final

multiple paths

single path reaches it only that will complete other will not.

Activity final node

when you reach node it completes activity

when no condition true activity stuck → so else condition used,

(iv) Decision and Merge Nodes:

one incoming arrow

multiple outgoing arrows.

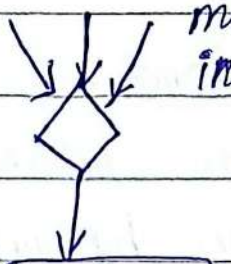
check tank level → [else] SoundAlert

[falls levels within tolerance]

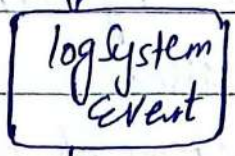
Guard condition

[ ] condition

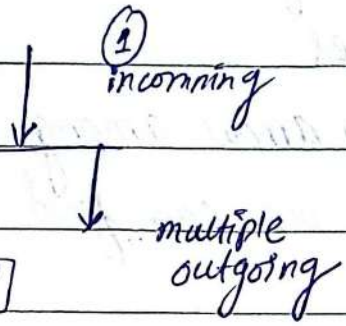
(v) Merge Nodes:



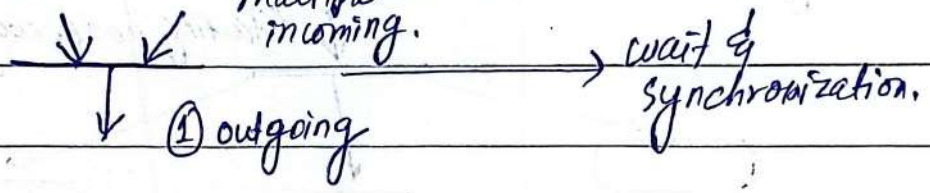
Forks, Joins
analogous to decision.



Forks:

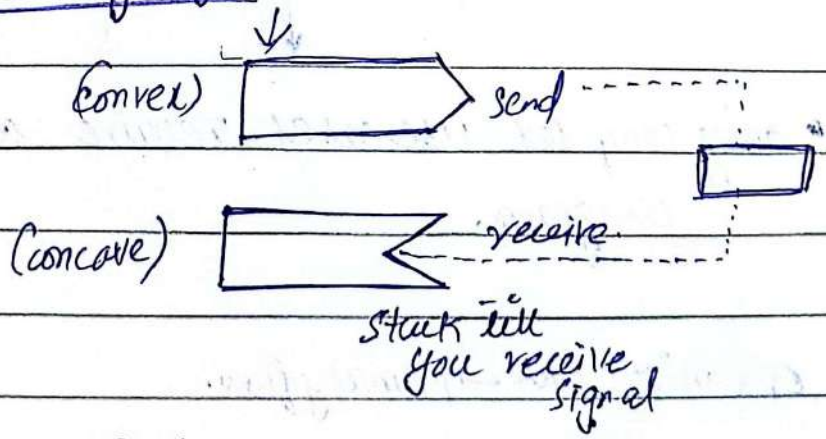


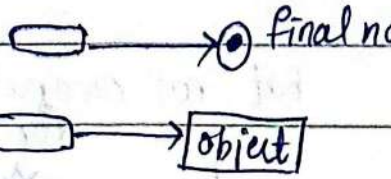
Join:



⇒ make activity diagram of complex (use case)

Sending & Receiving Signal:



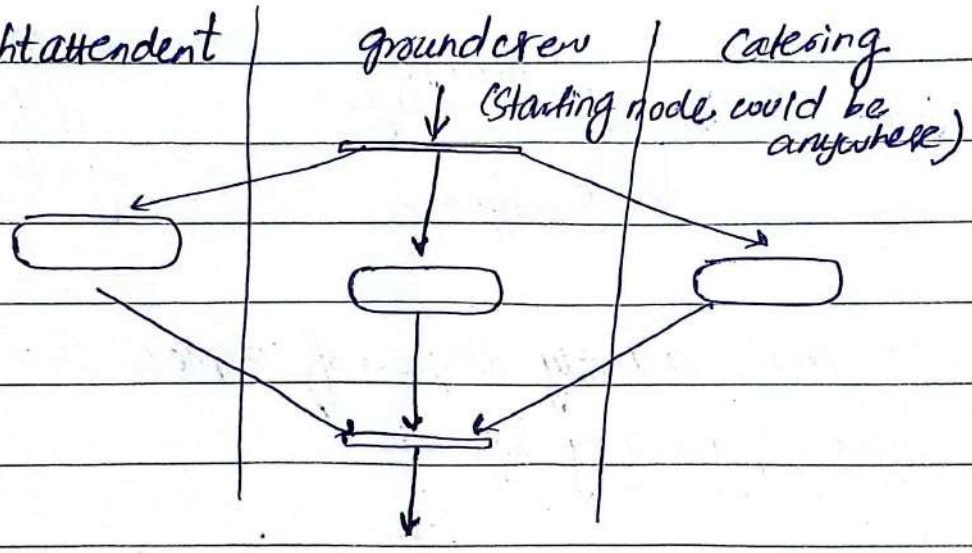


16/9/24 "3 ♥ Jatu

Swimlanes :-

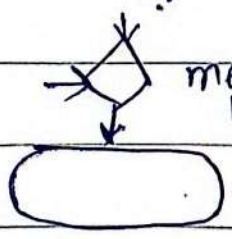
↳ columns which Tells a particular is done by which organization or classes.

- subsystems of a system
- lanes of classes
- If there is an arrow among one col. to other so, we can find interaction.

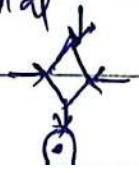


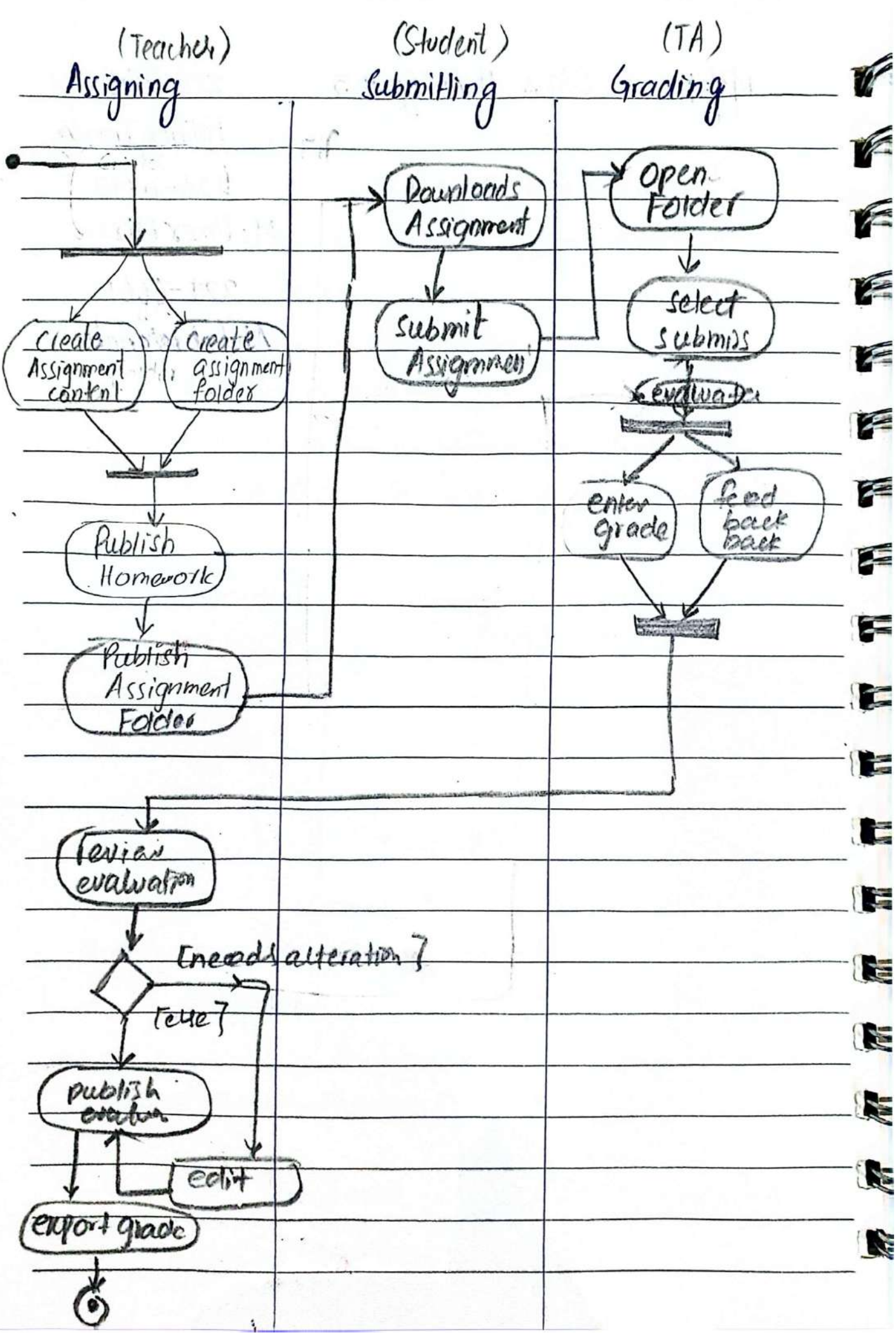
- only complex use cases require activity diagram.

Optimistic flow → primary flow.


merge node must for action or activity

but not compulsory for final




(Teacher)
Assigning
(Student)
Submitting
(TA)
Grading
Create Assignment content
Create assignment folder
Publish Homework
Publish Assignment Folder
Downloads Assignment
Submit Assignment
Open Folder
select submiss
evalua
enter grade
feed back back
review evaluation
[needs alteration]
[true]
publish evalua
edit
export grade

2/10/24

OOA → OOD

Analysis → Design

- requirement gather
- conceptual classes
- use case diagram.

transform these models to implementation specific logic.

- refine analysis model.
- define methods
- ✓ access specifiers/modifiers (private, public, protected)

refine relationships

weak aggregation
strong composition
specialized association

refined

Interaction and Collaborative Design:-

Sequence diagrams → how classes and objects within the system interacts

↓ dynamic behaviour

Design
- → functions in classes.

SOLID principles

Dynamic Modelling

→ Order of interaction is maintained

- interaction btw objects.
  - type (message passing)
    - call func of other object through message.

↑ fence formal → First object R →

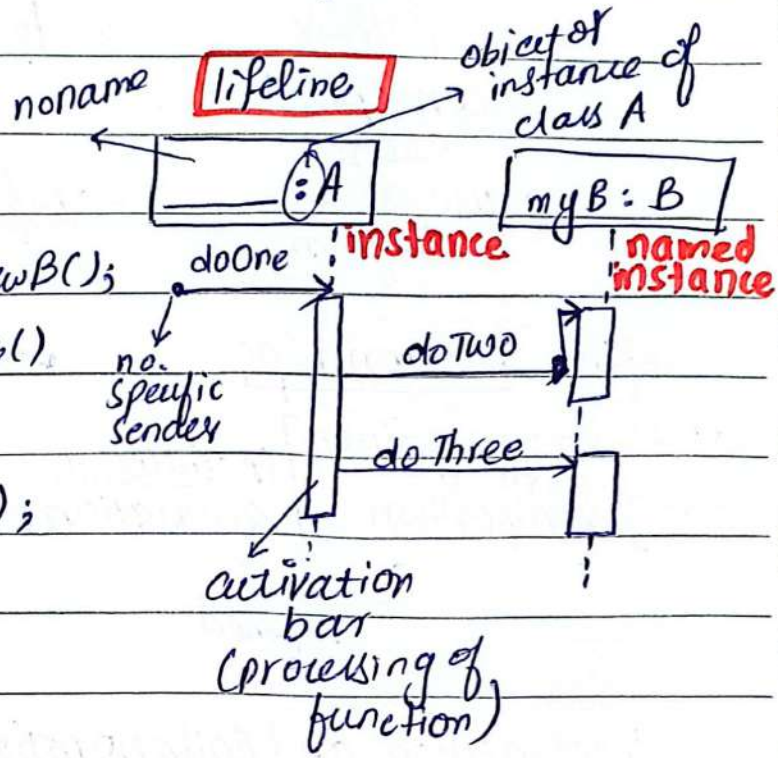
Design Sequence Diagram :→ separate for each **use case**

↓

we can do code translation through sequence diagram.

related classes or objects.

```
public class A
{
    private B myB = new B();
    public void doOne()
    {
        myB.doTwo();
        myB.doThree();
    }
    //....
}
```



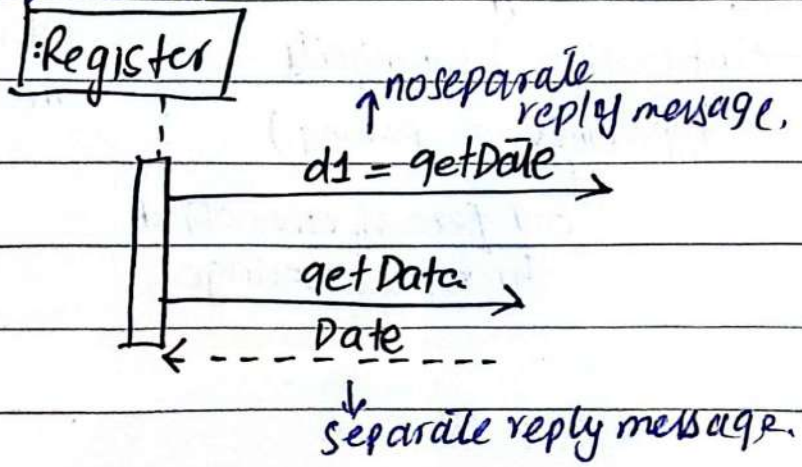
return = message (parameter : paraterType) : return Type

↳ functions return type.

d = getProductDesign (id: ItemId) : Product Design.

↓

write according to question requirement.



↑ filled arrows → filled line

Synchrous Message → line arrow.

Asynchronous

↓ replies are important and lifeline blocked till reply received

reply is optional

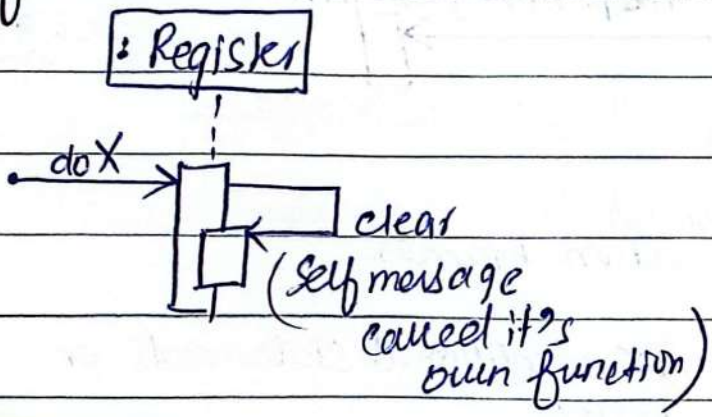
assume. response must come

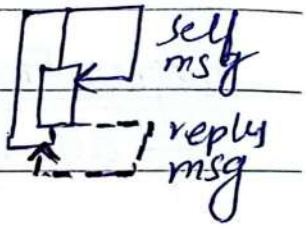
not necessary to make reply message

execution specific bar / focus of control / activation bar

③ names

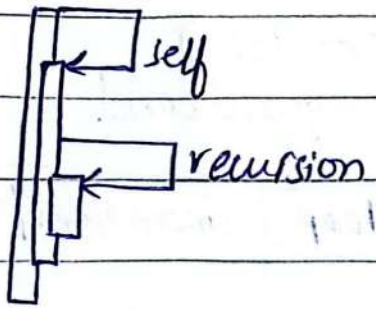
"self" or "this"





recursive

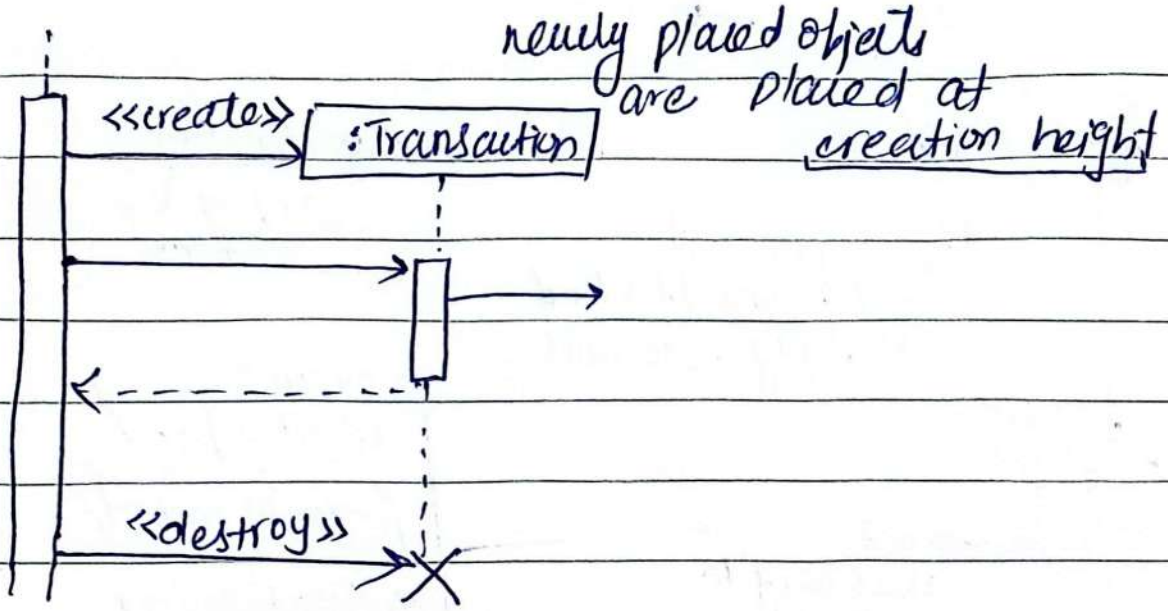
calls = 3 so 3 activation bars or write recursion



Create / Destroy messages → objects

temporarily exist → we use create msg

↳ Transient objects



Guard conditions:

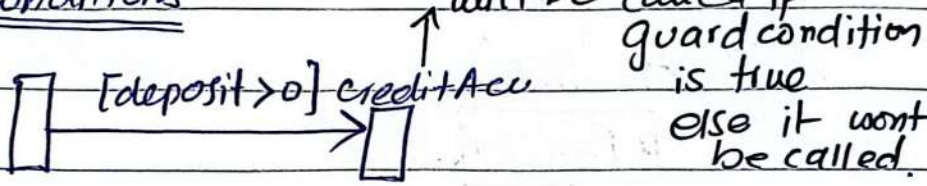
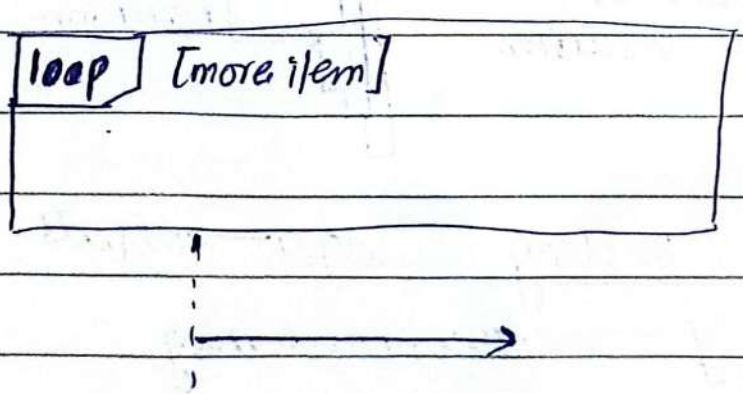


Diagram Frames:

↳ also known as Interaction Frames.

handle loops or conditional statements or concurrent tasks.

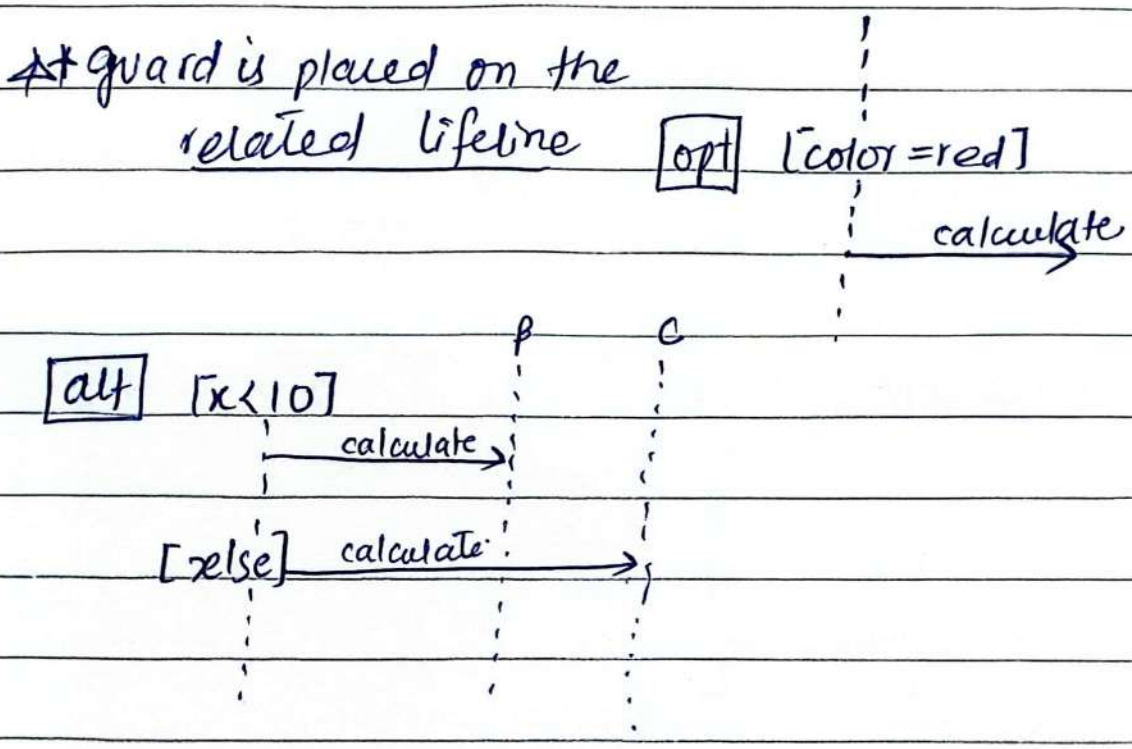
operator or label with guard cond.

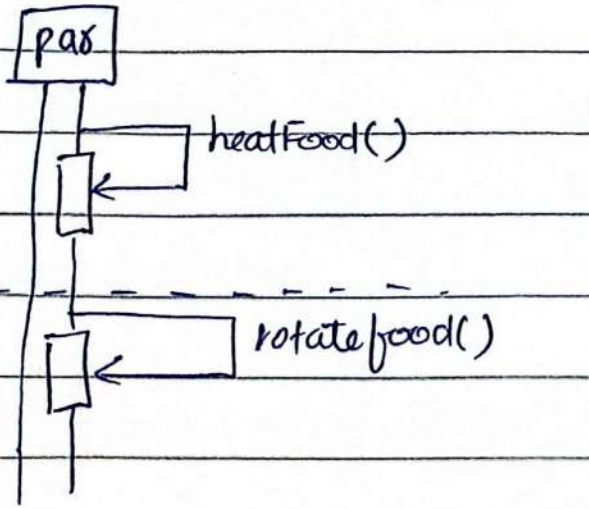


frame op

par	concurrent perform. (parallel)
alt	if else
opt	optional.
loop	

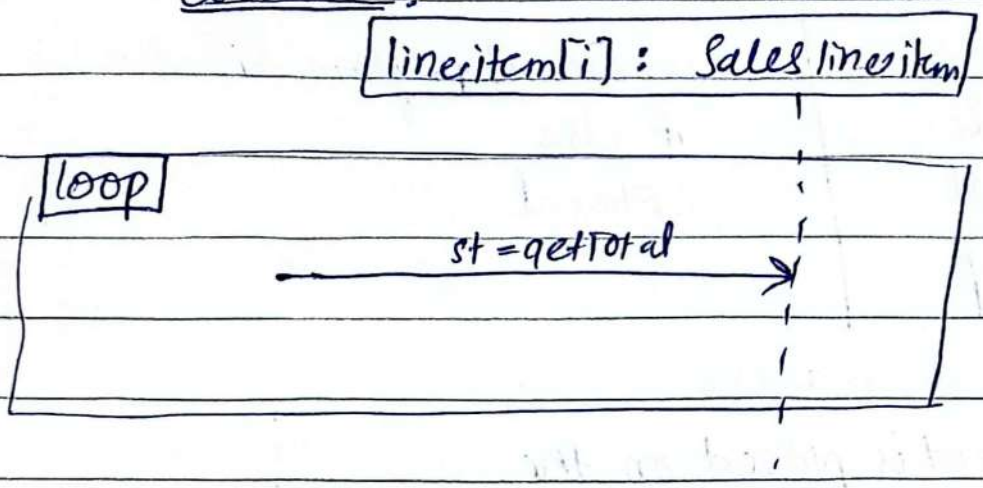
** guard is placed on the related lifeline





no of parallel tasks → no of make separate portions.

collection:



System Sequence Diagram :-

→ 1 use case scenario each.

(SSD) ↓

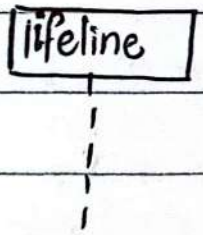
interaction between system and external actors.

↳ explains what a system does without explaining how it does.

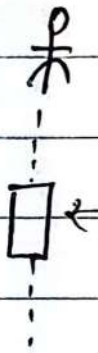
1) System (black box)
2) actors (+ initiating actor)
3) external system
4) msgs in & out of sys
5) sequence of msg.

① Lifeline

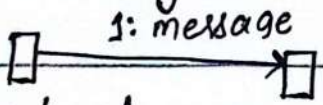
System → Actor



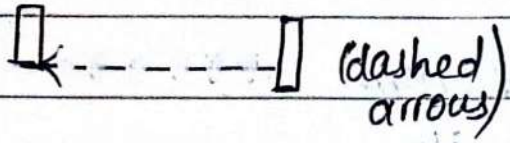
② Activation boxes (rarely used)



③ Call message



return/reply msg



Synchronous msg sig (solid arrow)

→ must wait for response.

(Blocking)

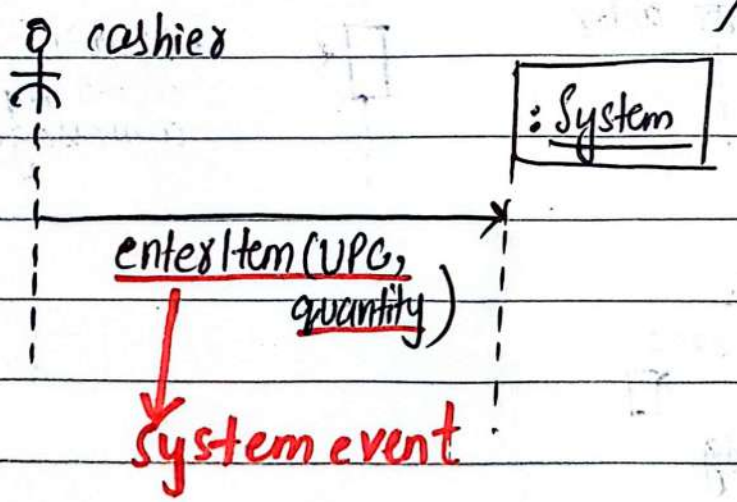
(both call + reply)

Asynchronous call : → (doesnot require response)

async return sig <- - - -

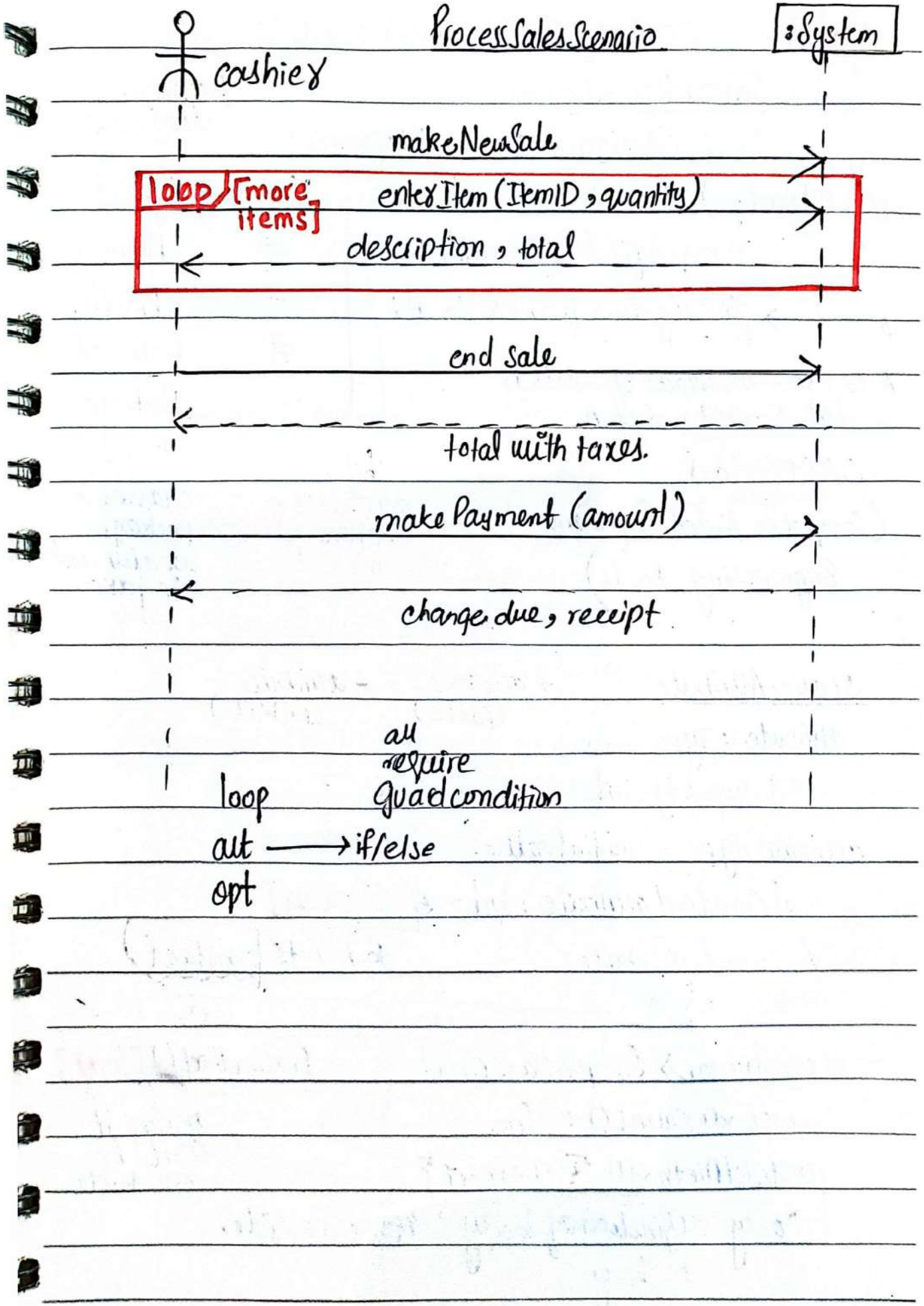
System event: external input generated by an actor (may include parameters)

make, start, enter, end etc.



bettername: enter item

worsename: Scan



all require guard condition

loop

alt → if/else

opt

9/10/24 OO Design Process and Models

Interaction diagram
↓
design Sequence Diagrams

package diagrams

enterItem(....)
↓unspecified parameters.

A ⟶ B (navigation from A to B)

A can see methods of class B but B can't see A.

CASE tool
(Computer Aided Software Engineering tools)

Visibility Symbols:

+	Public
-	Private
#	Protected
~	Package

↓ before attributes or before methods.

↓ multiple classes in package. usually used in java.

Static Attribute

attribute : Type
e.g quantity: int

attribute: type = initialvalue.

+ attribute (public)

- attribute (private)

final Constant attribute : int = 5 {frozen}
/derived attribute.

to tell its constant

+ <<constructor>> SampleClass(int)

methodReturns() : foo.

abstractMethod() {abstract}
↳ only definition of body its override
no body

finalmethod() {leaf}
↓ means it can't be override

SynchronizedMethod() [guarded]

↳ all a time only (1) thread can be executed.

A → B

⇒ navigation which means it has some B type attribute in A → reference attribute.

⇒ constructor can be added in methods or not.

→ Navigation is optional. Superior (major) or Subordinate rel

→ Subset class
→ provide info

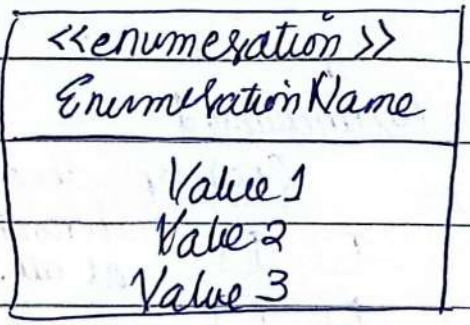
mandory association A → B

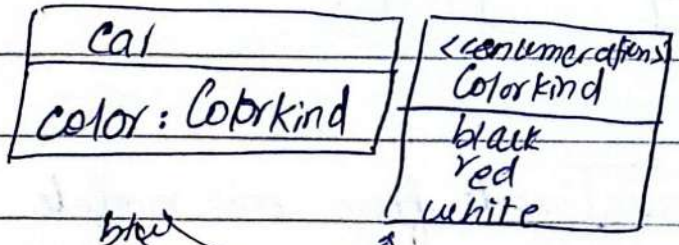
independent to dependent

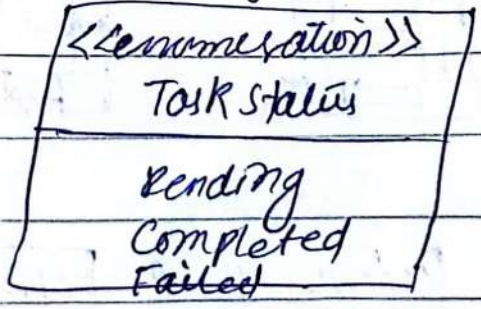
Customer → Sale.
(no customer no sale)

Enumeration classes :- (enum class)

↳ user defined data types, logically related things grouped together.







blue

these can be association but it will make complex so we avoid it.

14/10/24:- **Design Principles:**

↓⑥ dominant

(i) modularity M I I I A G

(ii) Interfaces

(iii) Info Hiding

(iv) Incremental development

(v) Abstraction

(vi) Generality.

① **Modularility:** → separation of concerns
- ↳ class
- ↳ attribute

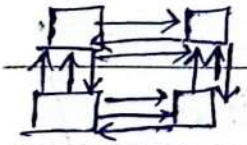
(i) **Coupling** → dependency of one module on another module.

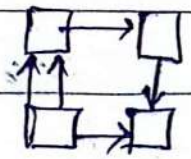
coupling should be less

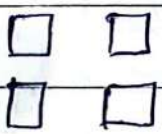
(a) highly coupled

(b) loosely

(c) uncoupled (no) dependency or interconnection at all.







"uses-a"

Dependency: (i) references made from one module to another

(ii) amount of data → parameters. passed from ----

(iii) amount of control that one module has on another module.

control execution, function envoke.

Modularity :- Types of Coupling

t m n
C C C S D U

(i) Content coupling: (very strong)
directly access or manipulate data of another module

(ii) Common coupling: instead of data in classes place it in [shared region.]

Shared data
if value is changed in one module then value will be changed in all modules.

increase of intensity of coupling

(iii) Control Coupling: one class control the behaviour of another class
A → B (control) execution

(iv) Stamp Coupling: pass complex data structures (e.g. class.) or unrelated info btw modules

(v) Data Coupling: only [data] values passed.

(2) Modularity: Cohesion → means dependency of module with its internal elements.

prefer * ⇒ more cohesive, more related pieces in module

(i) Coincidential cohesion (worst form)

↓ parts are unrelated to one another

CLTPCFS

unrelated funcs, processes or data are combined in same module for convience.

(ii) logical cohesion: → pieces relate on base of (logic)

(logic) payment → credit (diff functions)
↳ online

(iii) temporal cohesion (time): → put together things related with time.

↓ we are collecting time related things.

sale end
↳ inventory update
sale store
receipt generate

all these function grouped together in same class.

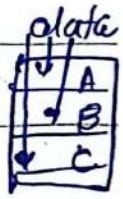
(iv) Procedural Cohesion: → in order to complete a task (procedure) we do some steps.

order processing
1) Check Inventory
2) Apply discount
⋮
↓ order of execution

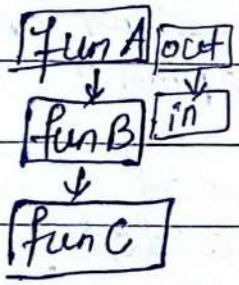
(v) Communication Cohesion: → place functions together that operate on same data

1) fetch sale data
2) cal total sale
3) Determin Top-Selling Products

↓ Get data = Sale.

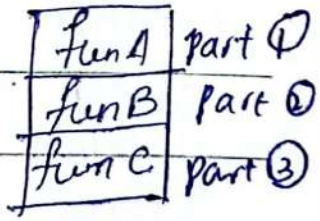


(vi) Sequential Cohesion: → output of one function is input of another function.



(vii) Functional Cohesion: (Best form)

↳ put together functions related to single, well-defined tasks.



CCCSD

CLT PCS F

2) **Interface**: hide implementation but tells services provided by it.

↓ Signatures (name) of public message only.

↓ provide:

require: parameters required by another module

purpose ①

specification :- documentation.

② preconditions (libraries need)

protocol (sequence, synchronization ④)

③ post condition (output?, error or exception?)

⑤ (non functional) requirement → Quality attribute ↓ security added efficiency added.

ball notation —O provide interface

Socket notation —C require interface.

Component logo.

3) **Information Hiding**: → eg private data members.

⇒ Interface → info hiding methode → loosely coupled.

→ module that hides an algo. may be functionally cohesive

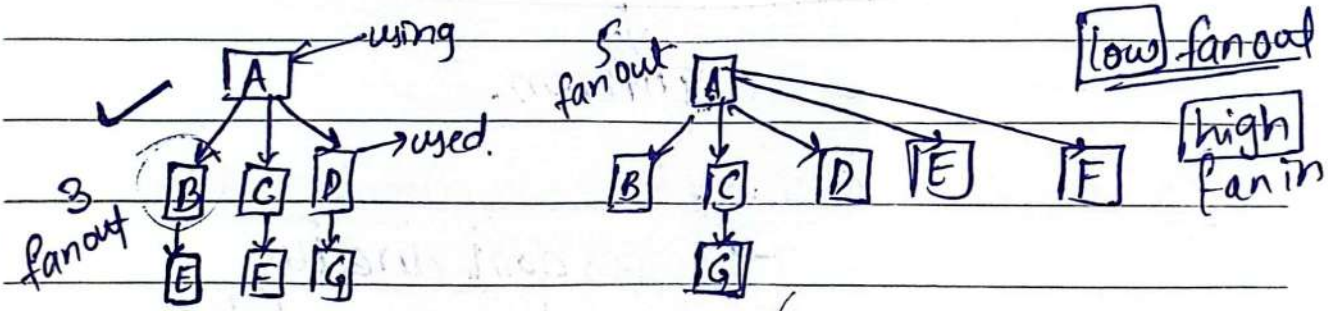
→ module that hides the sequence in which tasks are performed may be procedurally cohesive.

21/10/24 OCP opencodeprinciple

↳ class should be
open for extension
→ closed modification

4) Incremental Development:

□ nodes (software units) using → used.

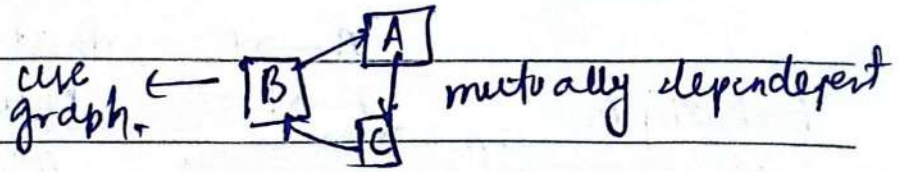


Fan-out: # of units used by node.

G (0) A(5) C(1) B(0)

Fan in: # of units that use a particula node

node G (1) node (D) (1).



↦ don't touch already existing code.

23/10/24

→ Open/Closed Principle

SOLID Principles → Liskov's Substitution principle.

compliment eachother.

Single Responsibility Principle

doesnot necessarily means.

Interface Segregation Principle:

2 interface ?

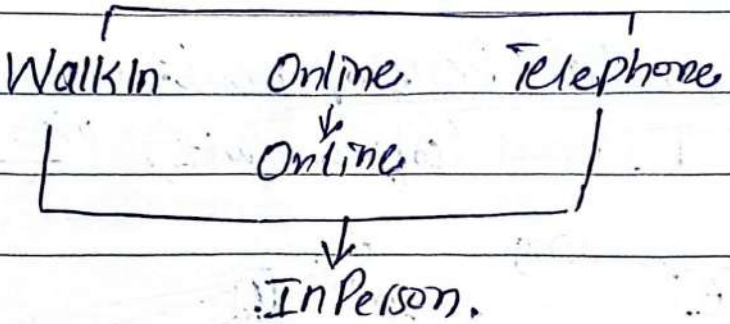
create interfaces or abstract classes

→ Open/close principle followed

→ don't force client or class to implement interface it doesnot need.

if only one general purpose interface, code violated.

2 diff users need diff. functionalities make or generate customizable interfaces.

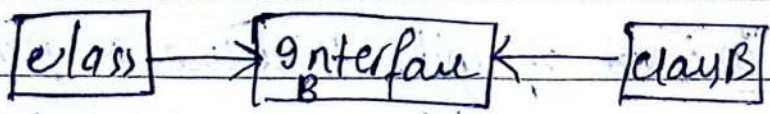


Dependency Inversion Principle:

HL classes use LL classes to complete it's own work.

→ Classes don't directly depend upon eachother

→ make or introduce interface through which they communicate.



A → B
A → I ← B

that's why known as dependency invert principle

book obj in Shelf cause ISP violation.

hollow arrow.

implement interface

(shelf-book)

verb in past tense or to onset of some condition

Event, state, transition.

UML : State Diagram

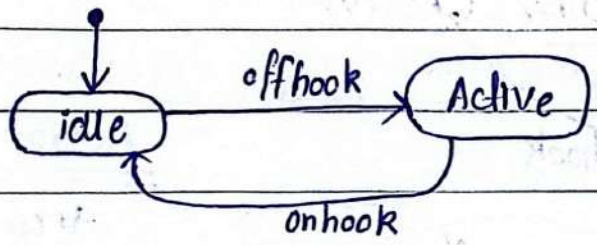
Class → structure diagrams

use case, activity, state → behaviour diagrams

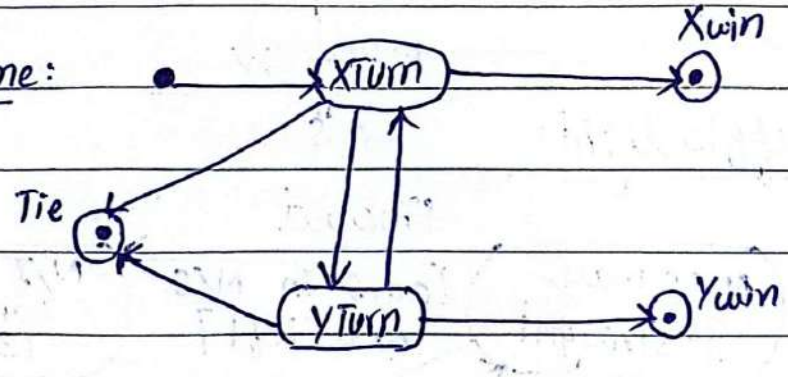
Sequence → Interaction diagrams. (dynamic picture)

Statechart diagram or State machine diagram

Telephone:



TicTacToeGame:



Event

↳ change event (boolean F/T expression)

e.g when (roomtemp < heating pt)

↳ signal event

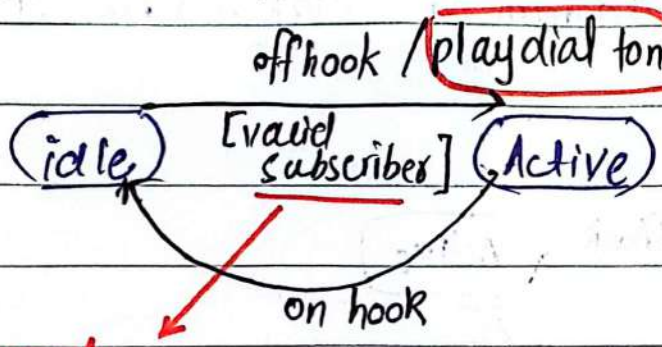
↳ timeout event (involve time)

e.g after(10sec)

when (date = 10/10/24)

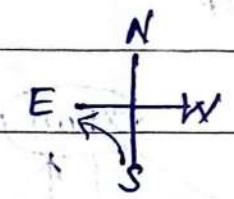
State: → suffix 'ing' e.g (dialing)
↳ duration of some event e.g (Powered)

transition
↳ event occurs
(unless an optional [guard condition] causes the event to be ignored)
↳ only fires when event occurs and it's true.

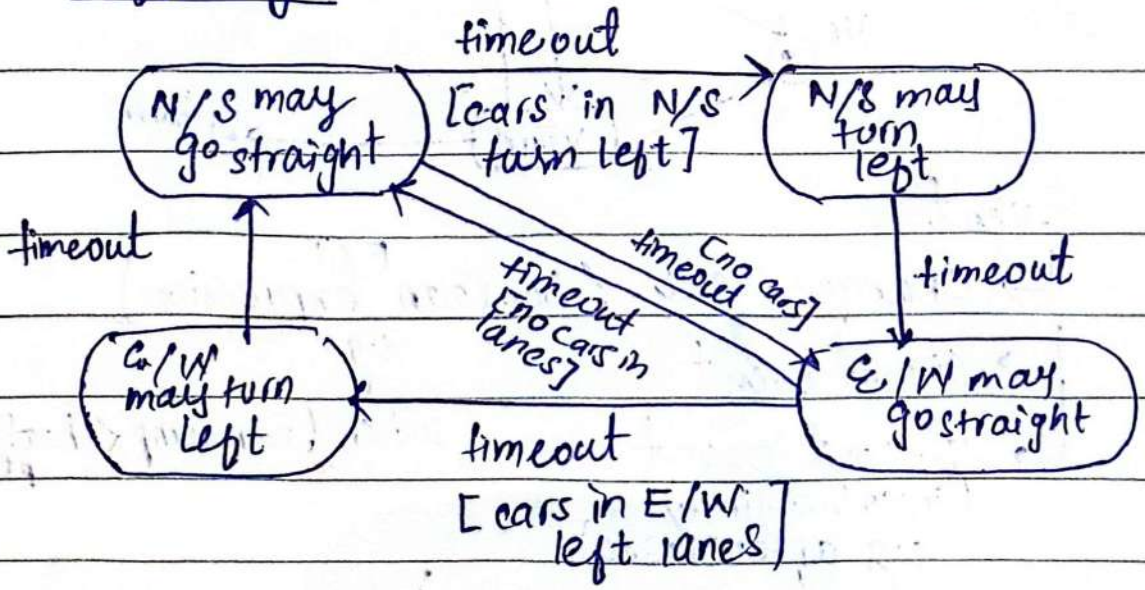


Transaction Action: transition may cause an action to fire.

guard condition.



Traffic light:



effect → response to behaviour that is executed in response to an event.

activity that can be invoked by no of effects.

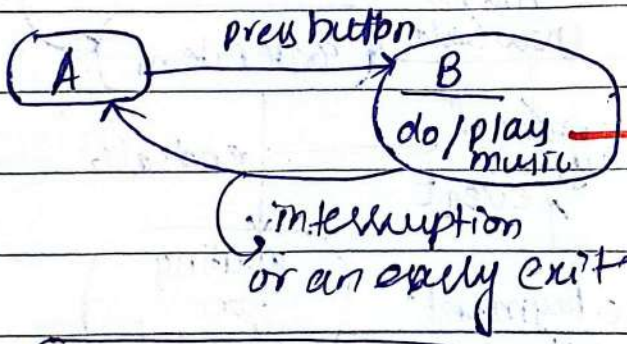
Activity:- / name or description of act

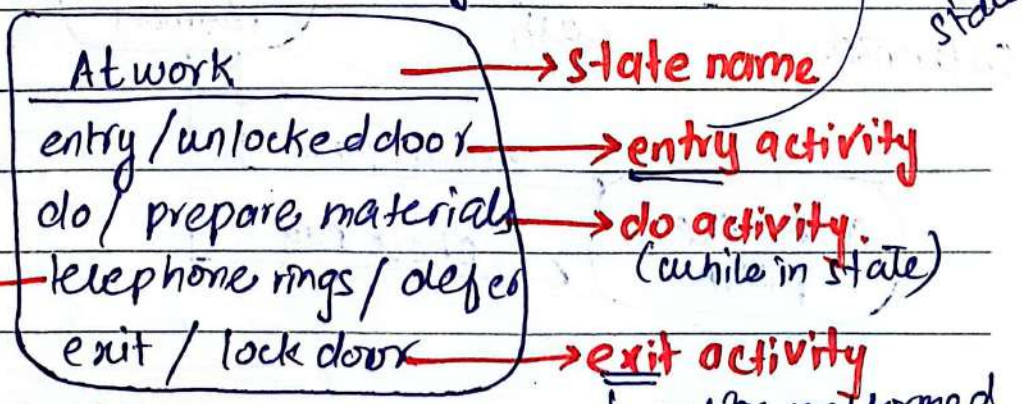
e.g (idle) —event: R button down / display pop-up→ Activity/Action

Do Activity: → continues for "do/name" extended time period.

e.g Photocopy:

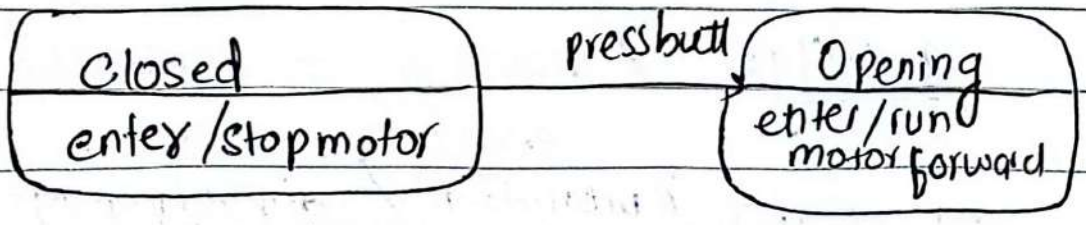
PaperJam
do/flash warning light



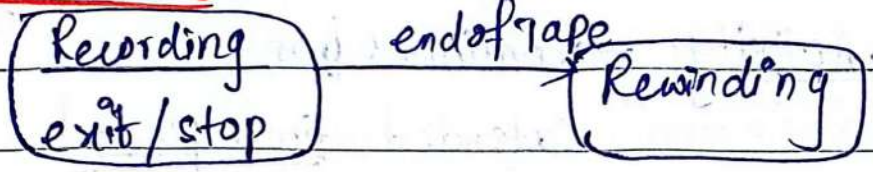


(event that does not trigger any state trans but remain in the event pool)

Enter Activity:

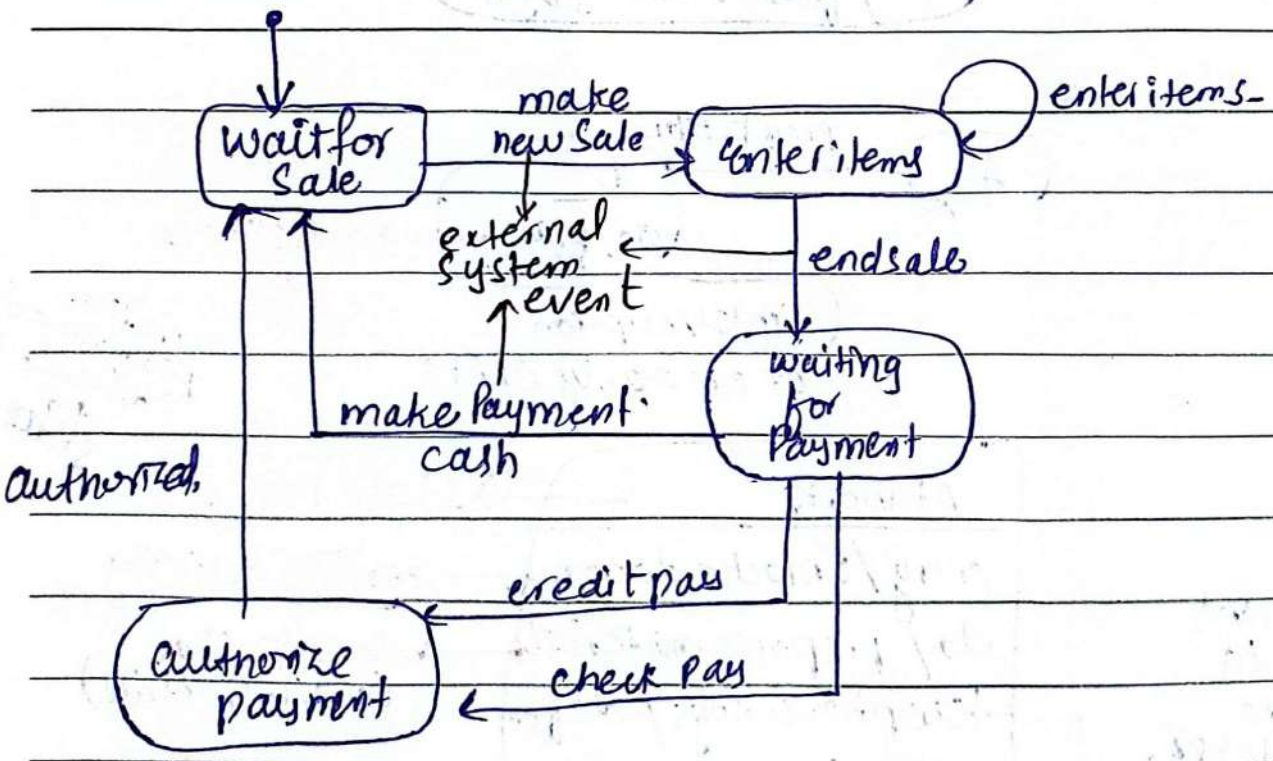


Exit Activity:



e.g

Process Sale:

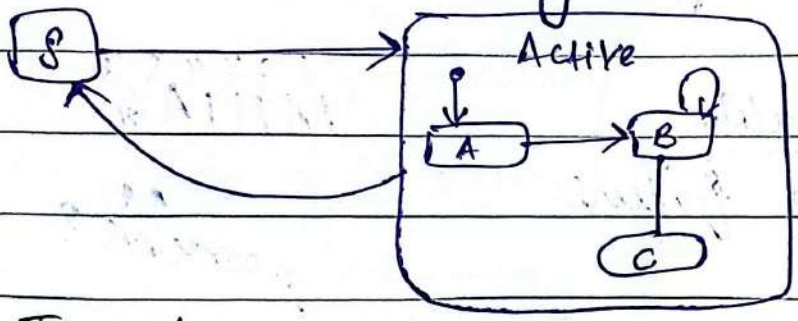


Event Types:

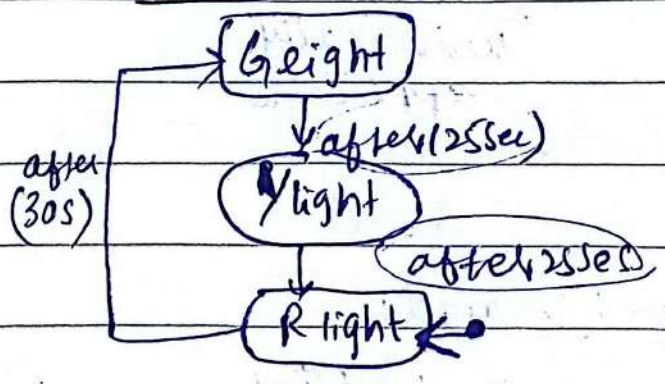
- External event (System event)
  - Cause by actor outside our system boundary
- Interval event
  - Caused by someone inside system boundary
- Temporal event
  - Occurence of specific date or time
  - e.g make pay op must occur after end sale

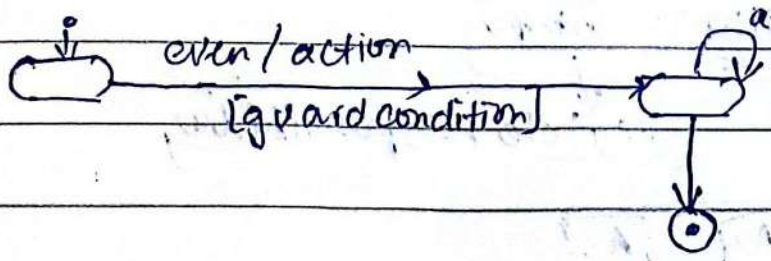
Nested states:

State allowing Substates

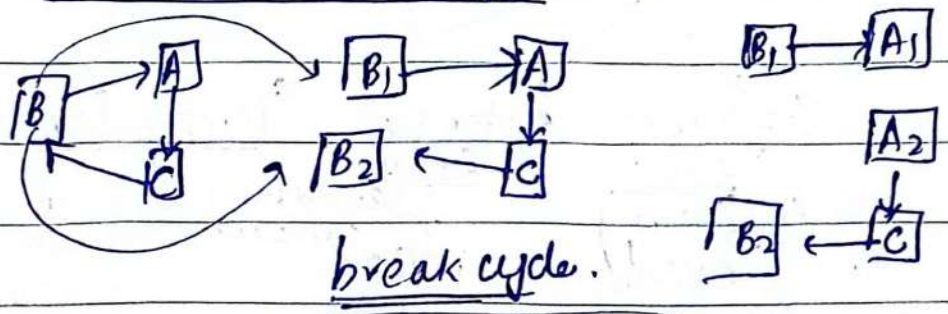


Timeout:





4) incremental development:



break cycle.

* (Sandwiching)

(A) one unit decompose → A1 (2) units
→ A2

5) Abstraction:

necessary details

6) Generality

useful some future

1 2 3 4 5 6
M I I I A G

contexts ?

□ node / software unit → general
universally applicable

(i) Parameters

any data op

(ii) Preconditions Removed:

↓ use donot restrict.

(ii) Postconditions: Simply

split A → divide responsibility

A1 A2 A3

SOLID :

① SRP (Single Responsibility Principle)

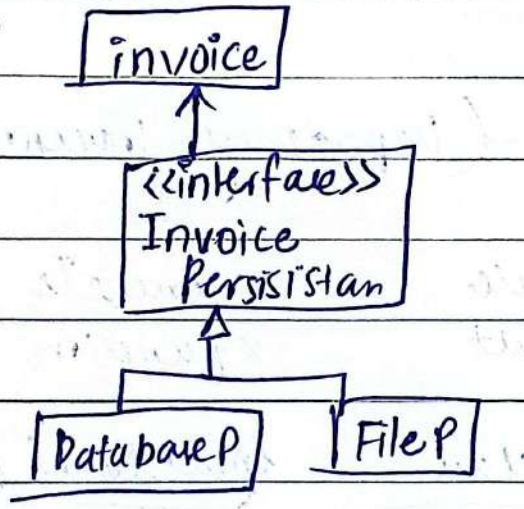
→ class do one thing.

one and only one actor.

② OCP (Openclose Principle) → interfaces (service)
→ abstract classes (necessary data)

classes open for extention

claves closed for moclificatwn



③ (Lisvok Substitution LSP Principle)

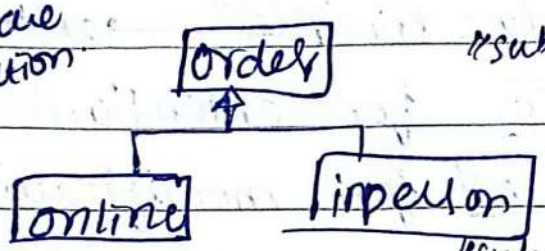
Subclasses should be suitable for base classes.

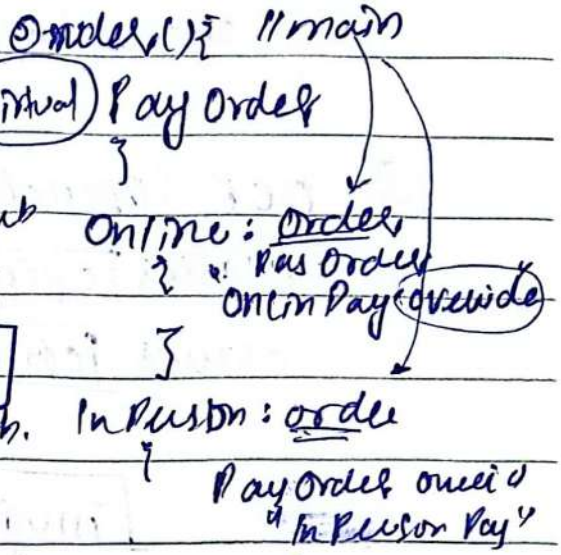
④ (Interface Segregation Principle)

ISP (separate)

throw logic_error(" ")

⇒ Interface Pollution





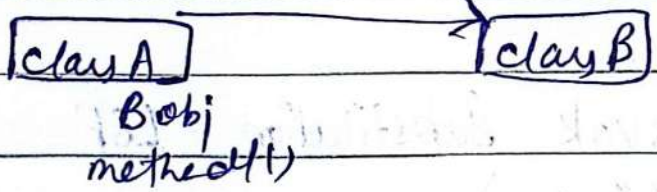
⑤ DIP (Depedency Inversion Principle)

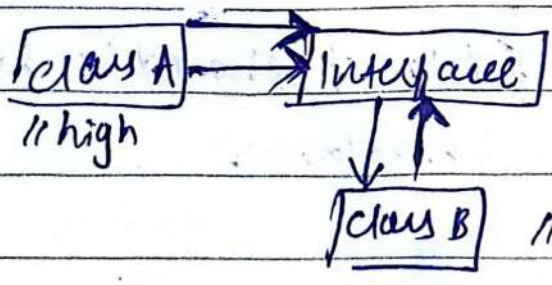
↓

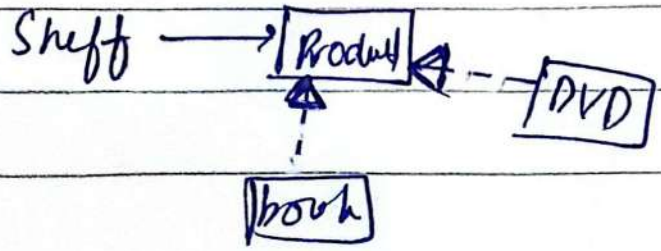
interface × concrete

abstract × functions.

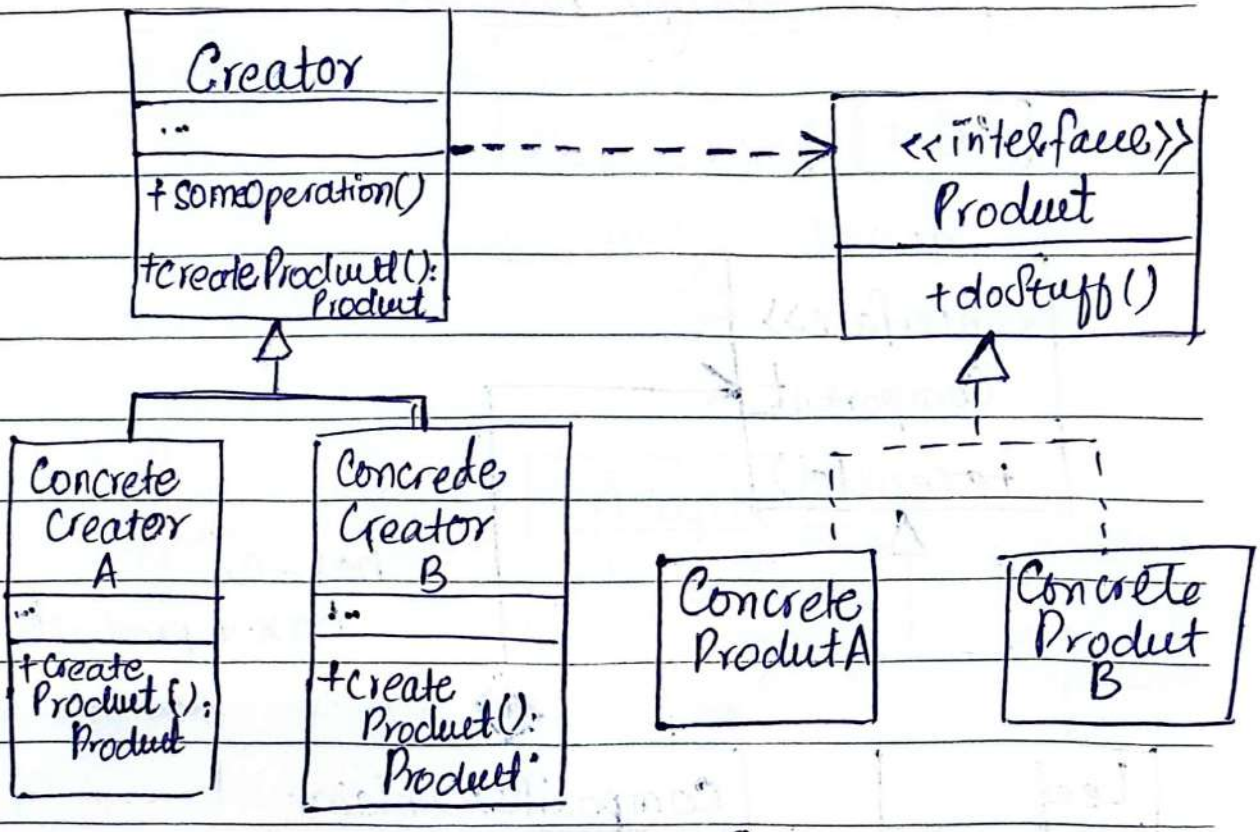
High level class → low level class.







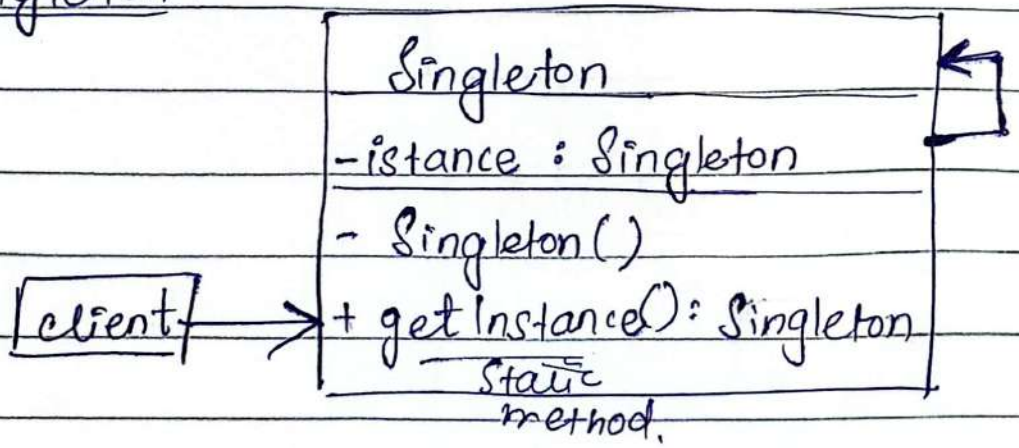
① Factory Method Structure :- Creational



Creator ---- Product

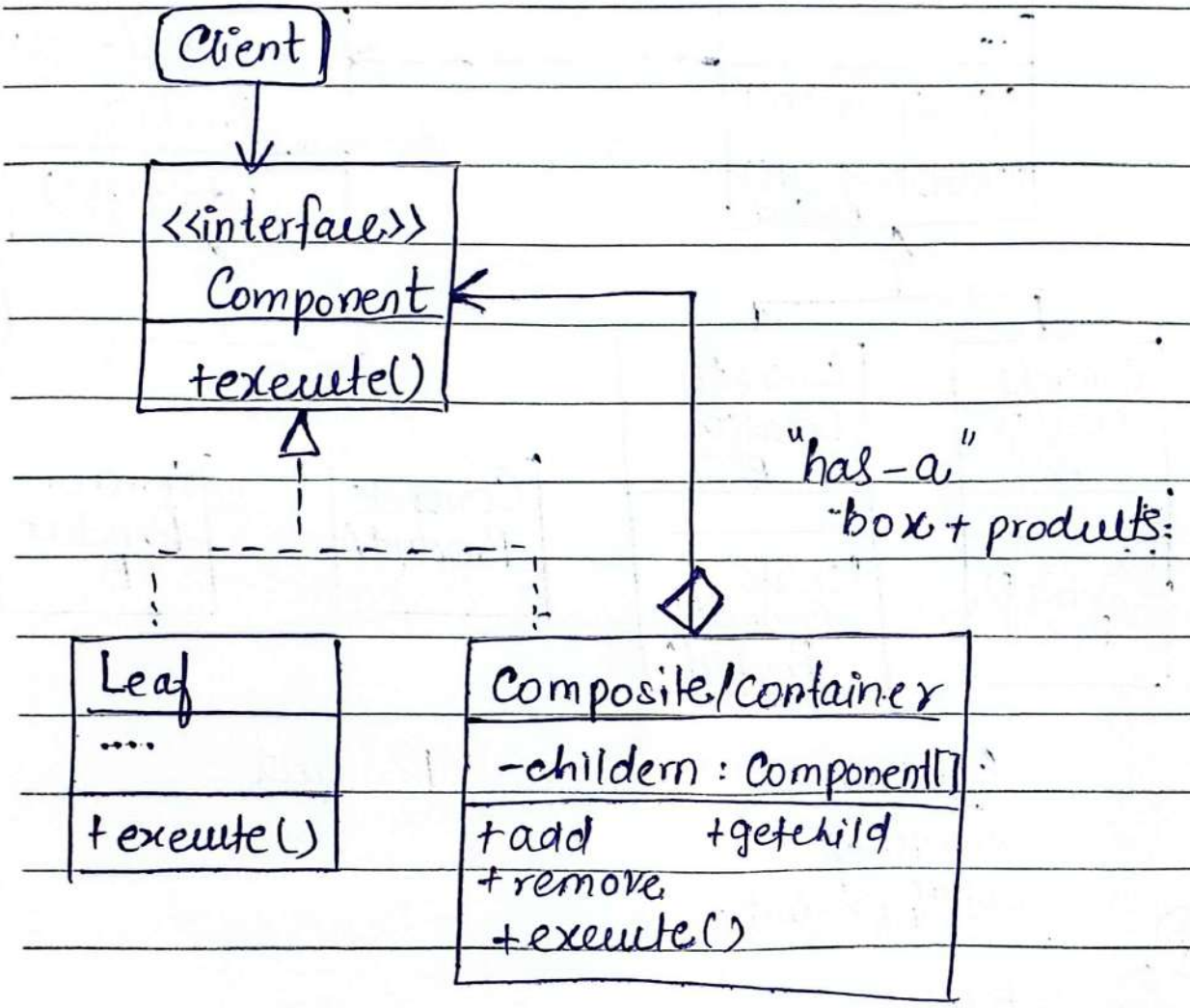
Creator
⟸
consist of
obj of Product.

② Singleton :-



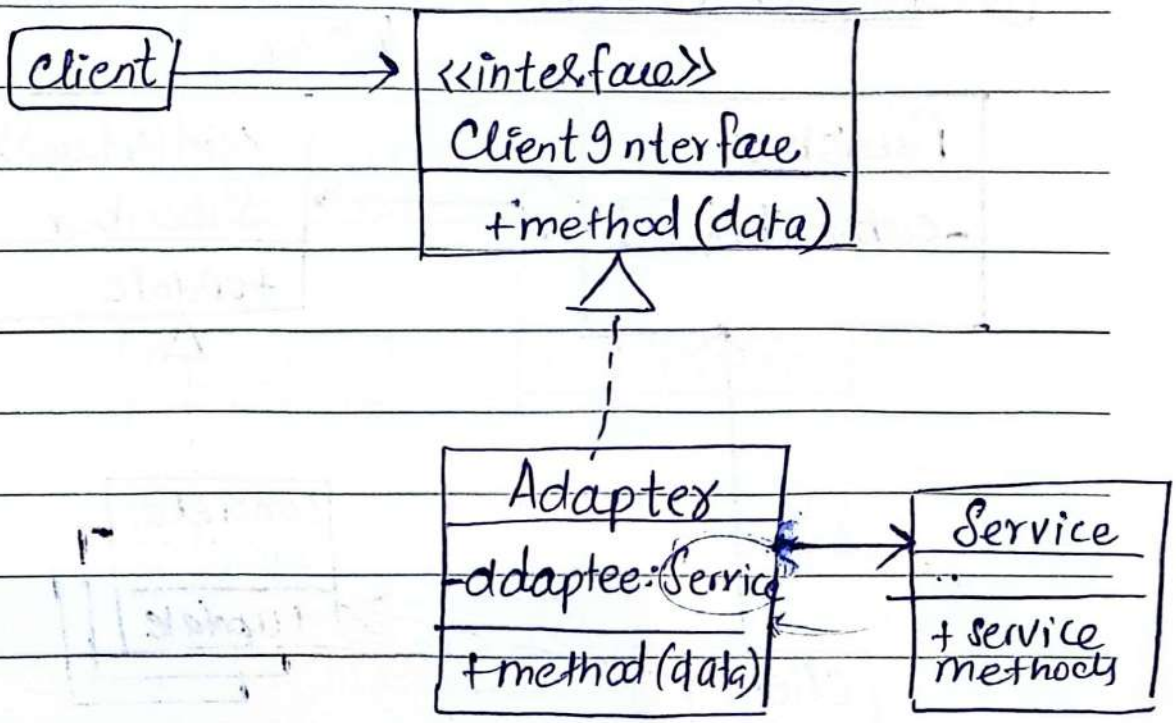
Object tree Structural

① Composite Design Pattern:

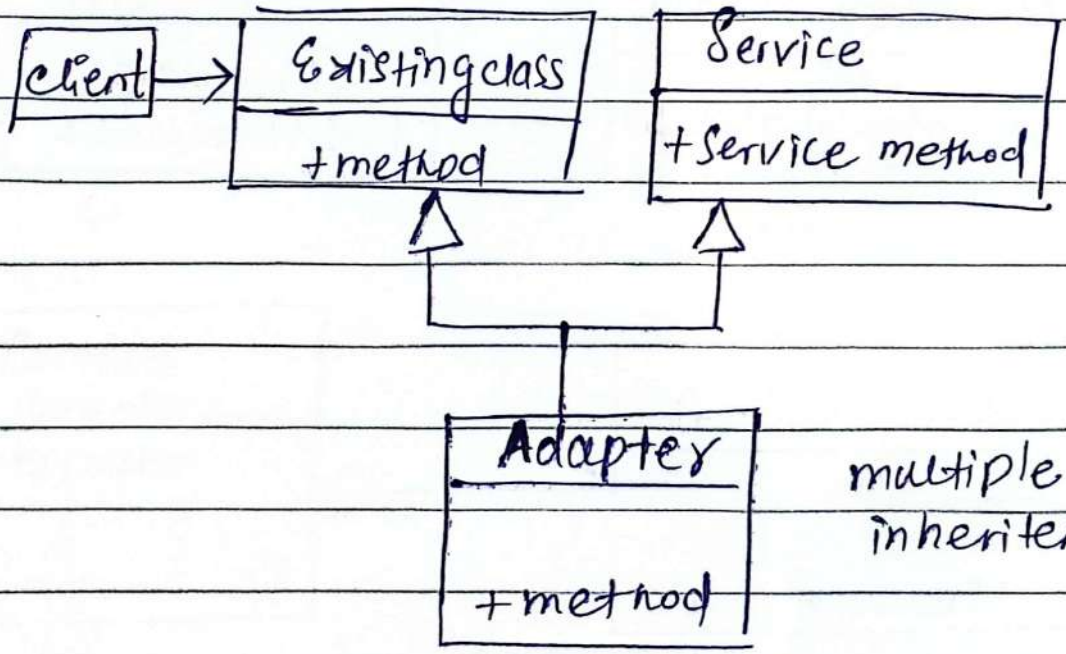


Adapter class Patter
wrapper

② (Object Adapter) :-



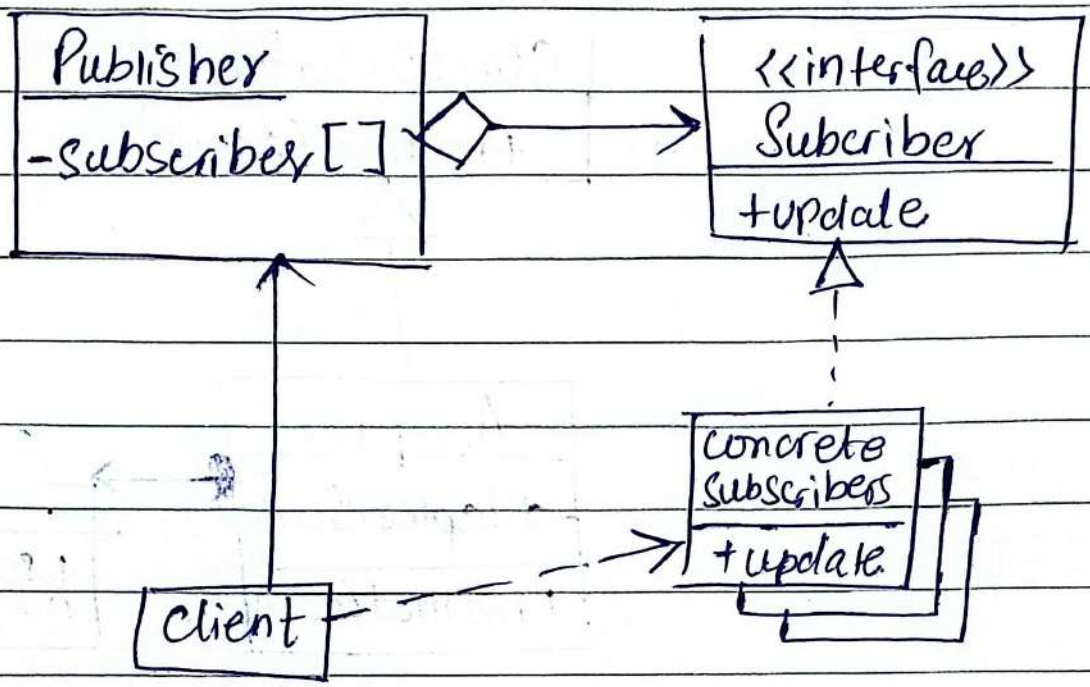
(Class Adapter) :-



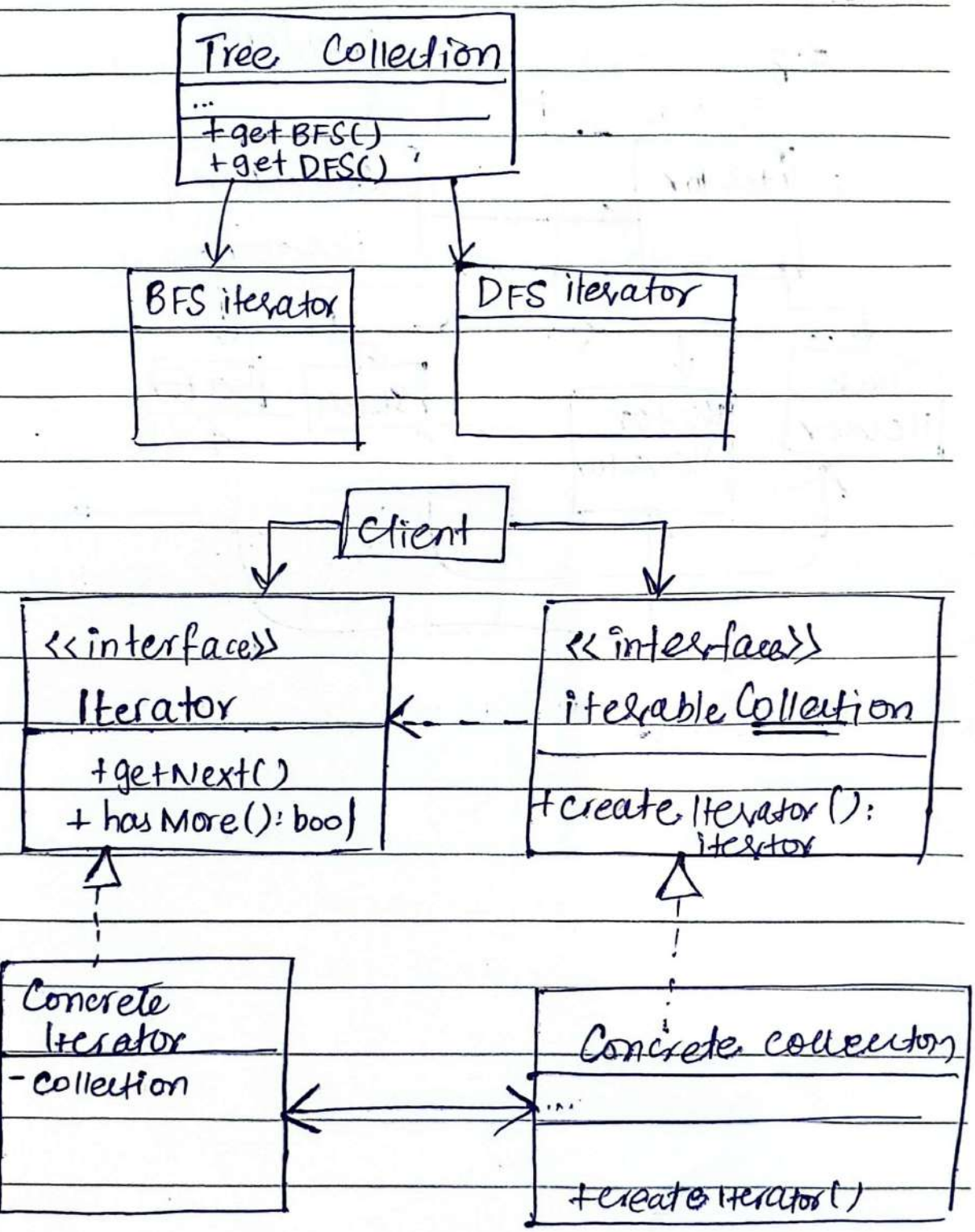
multiple inheritence

Behaviour

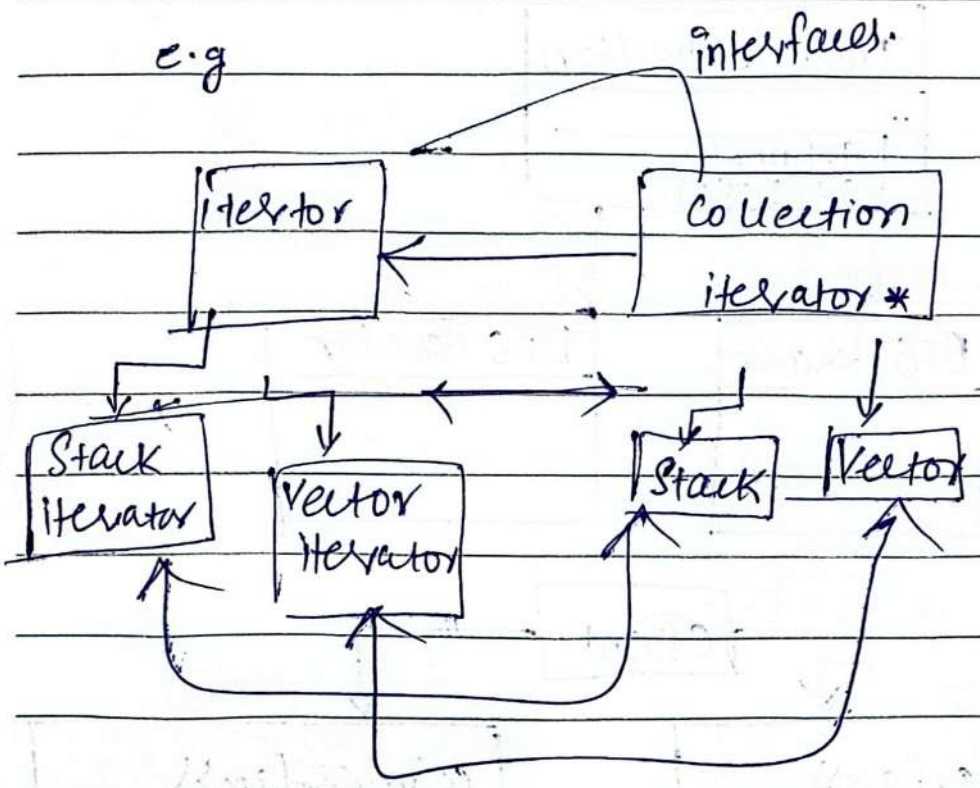
① Observer Pattern Event- Subscribers listner



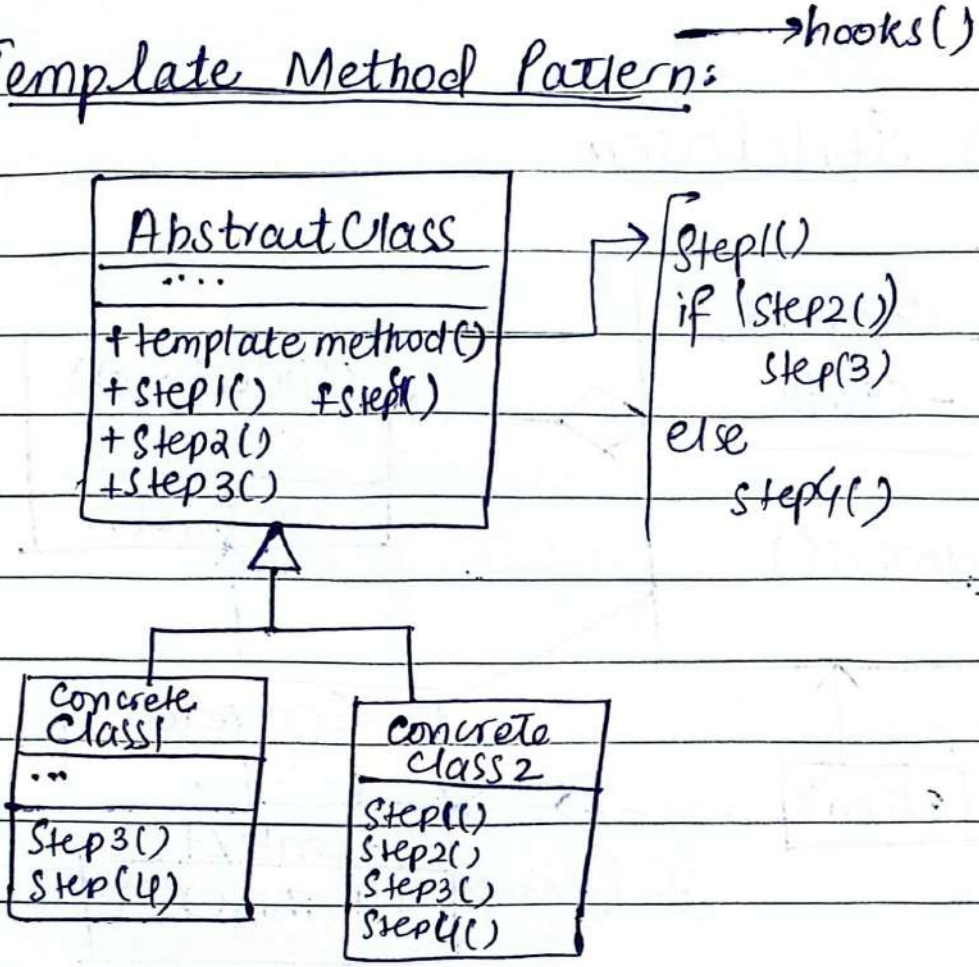
② Iterator Pattern:



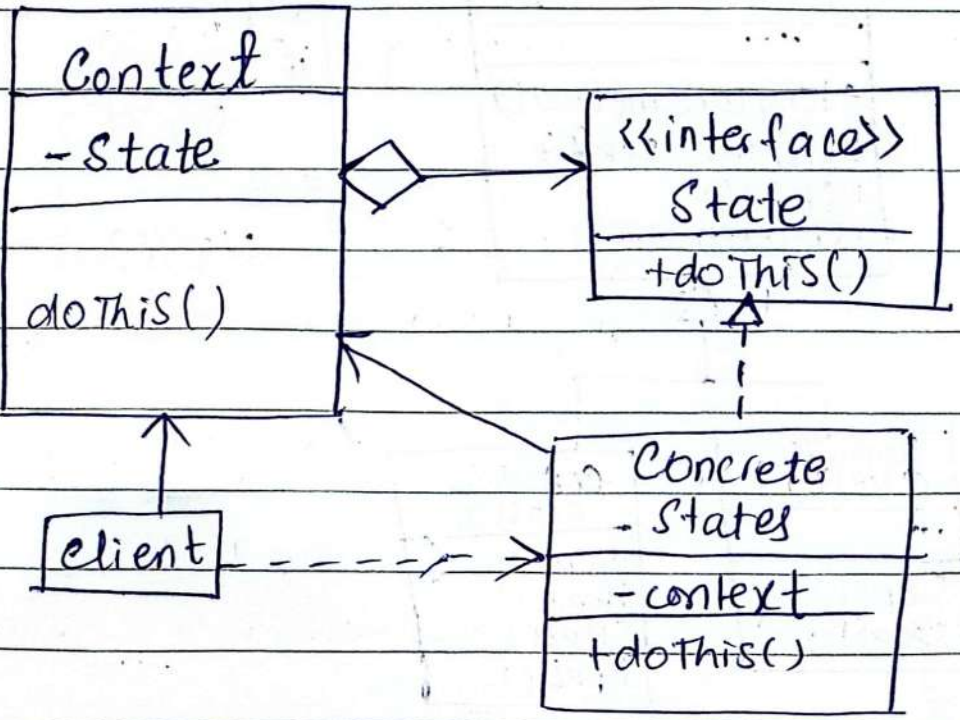
e.g


interfaces.
Iterator
Collection
iterator *
Stack
Iterator
Vector
Iterator
Stack
Vector

Template Method Pattern:



⑦ StatePattern



⑧ LCOM (Lack of cohesion in methods)

↓

How closely related the methods in class are

method pairs → connected
→ disconnected

LCOM = # of disconn pairs − # of conn pairs

⇒ LCOM > 0 lack of cohesion

⇒ LCOM ≤ 0 good cohesion

CK Metrics:

① Cyclomatic Complexity:

↓

# of linearly independent paths in it.

→ $V(G) = E - N + 2$

E = # of edges

N = # of nodes.

$= 10 - 10 + 2 = \boxed{2}$ → 2 independent paths.

② WMC

③ White Box testing

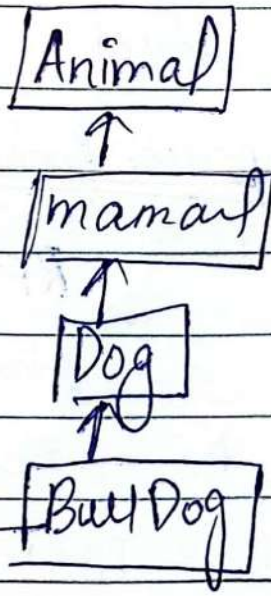
④ DIT (depth of inheritence tree)

lenght of longest path root → leaf.

DIT ↑↑ behaviour more difficult ↑↑

complexity ↑↑

reuse ↑↑



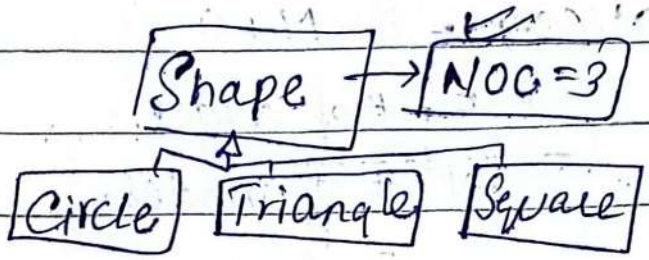
DIT ← BullDog

= 3

(5) NOC Numbers of children
intermediate subclass

Higher NOC ↑↑ → over generalization.

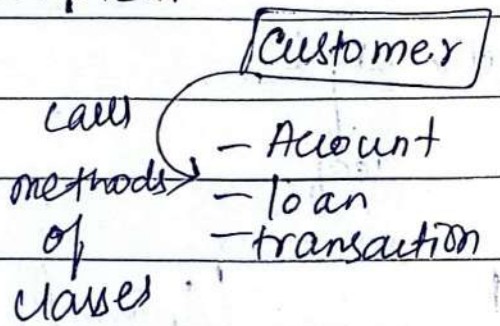
Complexity ↑↑



(6) CBO Coupling between object classes:

no of classes to which given class is coupled.

CBO ↑ complexity ↑ maintain ↓



(7) Response from class (RFC) → should be kept low.

$$RFC = M + R$$

M → # of methods in class

R → # of remotes that are called by methods of class

RFC ↑ testing effort ↑